

# Cheat\_Sheet

## 排序算法

归并排序（可用于求逆序对数）

```
# start--mid 和 mid+1--end 都是sorted list
def Merge(a,start,mid,end):
    tmp=[]
    l=start
    r=mid+1
    while l<=mid and r<=end:
        if a[l]<=a[r]:
            tmp.append(a[l])
            l+=1
        else:
            tmp.append(a[r])
            r+=1
    # 以下至少有一个extended了空列表
    tmp.extend(a[l:mid+1])
    tmp.extend(a[r:end+1])
    for i in range(start,end+1):
        a[i]= tmp[i-start]

# 二分
def MergeSort(a,start,end):
    if start==end:
        return

    mid=(start+end)//2
    MergeSort(a,start,mid)
    MergeSort(a,mid+1,end)
    Merge(a,start,mid,end)

a=[8,5,6,4,3,7,10,2]
MergeSort(a,0,7)
print(a)
```

快速排序

```
def quicksort(arr, left, right):
    if left < right:
        partition_pos = partition(arr, left, right)
        quicksort(arr, left, partition_pos - 1)
        quicksort(arr, partition_pos + 1, right)

def partition(arr, left, right):
    # 最右端元素作为基准元素，i,j是两个指针，通过两个元素的交换实现
    # 基准元素左右分别小于、大于他本身。
    i = left
    j = right - 1
    pivot = arr[right]
    while i <= j:
        while i <= right and arr[i] < pivot:
            i += 1
        while j >= left and arr[j] >= pivot:
            j -= 1
        if i < j:
            arr[i], arr[j] = arr[j], arr[i]
    if arr[i] > pivot:
        arr[i], arr[right] = arr[right], arr[i]
    return i

arr = [22, 11, 88, 66, 55, 77, 33, 44]
quicksort(arr, 0, len(arr) - 1)
print(arr)

# [11, 22, 33, 44, 55, 66, 77, 88]
```

## 全排列

递归生成所有排列（交换法，原地无需visited数组）

```
def permutations(arr):
    res = []
    def backtrack(start):
```

```

    if start == len(arr):
        res.append(arr[:])
        return
    for i in range(start, len(arr)):
        arr[start], arr[i] = arr[i], arr[start]
        backtrack(start+1)
        arr[start], arr[i] = arr[i], arr[start]
    backtrack(0)
    return res

```

### 下一个字典序排列 (next\_permutation)

算法：从右往左找第一个升序对  $arr[i] < arr[i+1]$ （找不到则已是最大排列）；再从右往左找第一个  $arr[j] > arr[i]$ ，交换；最后反转  $i+1$  到末尾。 $O(n)$ 。

```

def next_permutation(arr):
    n = len(arr)
    i = n - 2
    while i >= 0 and arr[i] >= arr[i+1]:
        i -= 1
    if i < 0:
        return False
    j = n - 1
    while arr[j] <= arr[i]:
        j -= 1
    arr[i], arr[j] = arr[j], arr[i]
    left, right = i+1, n-1
    while left < right:
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1
    return True

# 按字典序打印所有排列
s = list("abc")
while True:
    print(''.join(s))
    if not next_permutation(s):
        break

# 求给定字符串的下一个排列
def next_permutation_str(s):
    lst = list(s)
    if next_permutation(lst):
        return ''.join(lst)
    return None

print(next_permutation_str("acb")) # "bac"

```

Python标准库可直接用 `from itertools import permutations`，但手写 `next_permutation` 能原地迭代生成，常数更小。

## 单调栈

奶牛排队，寻找右侧第一个小于i的索引

题目要求：N为数组长度，hi为数组元素，求最长满足条件子序列，子序列的左边界是该子序列的严格最小值，右边界是该子序列的严格最大值。

```

N,res=int(input()),0
hi=[int(input()) for _ in range(N)]
# left[i]是i左边第一个不小于他的元素的索引，right[i]是i右边第一个不大于他的元素的索引。
# 容易知道，对于指定的i，如果i作为右端点，left[i]是左端点的一个上界，反之同理。
left,right=[-1 for _ in range(N)],[N for _ in range(N)]
stack1,stack2=[],[]

for i in range(N-1,-1,-1):
    while stack1 and hi[stack1[-1]]>hi[i]:
        stack1.pop()
    if stack1:right[i]=stack1[-1]
    stack1.append(i)

for i in range(N):
    while stack2 and hi[stack2[-1]]<hi[i]:
        stack2.pop()
    if stack2:left[i]=stack2[-1]
    stack2.append(i)

for i in range(N):
    for j in range(right[i]-1,i,-1):
        if left[j]<i:
            res=max(j-i+1,res)
            break

```

```
print(res)
```

## 栈的应用：合法出栈序列（M30637）

**题意：**给定原始字符串x（字符依次入栈），判断给定序列seq是否可能是x的一个合法出栈序列。

**算法：**双指针+模拟栈。对seq中每个字符c，从x中持续push直到栈顶等于c，然后pop。若中途x耗尽仍找不到c，则非法。

| 输入： | 输出：                                            |
|-----|------------------------------------------------|
| abc | YES (push a pop a, push b pop b, push c pop c) |
| abc | YES (push abc, pop c, pop b, pop a)            |
| bca | YES (push ab, pop b, push c, pop c, pop a)     |
| cab | NO (要先输出c需push abc, pop c后栈是[a,b]，要a须先pop b)   |

```
import sys

def is_valid_pop(x, seq):
    if len(seq) != len(x) or set(seq) != set(x):
        return False
    stack, i = [], 0
    for c in seq:
        while not stack or stack[-1] != c:
            if i >= len(x):
                return False
            stack.append(x[i])
            i += 1
        stack.pop()
    return True

data = sys.stdin.read().split('\n')
x = data[0].strip()
for line in data[1:]:
    line = line.strip()
    if not line:
        continue
    print("YES" if is_valid_pop(x, line) else "NO")
```

**复杂度：**每个字符最多入栈/出栈一次，单次判断O(n)。

## 后序表达式求值

从左侧先后弹出两个数字a,b一个算符@，计算a@b，再放回左边。

样例输入

```
3
5 3.4 +
5 3.4 + 6 /
5 3.4 + 6 * 3 +
```

样例输出

```
8.40
1.40
53.40
```

```
def cal(a,b,operate):
    if operate=="+":return a+b
    if operate=="-":return a-b
    if operate=="*":return a*b
    if operate=="/":return a/b

from collections import deque
n,operators=int(input()),('+','-','*', '/')
raw=[deque(map(str,input().split())) for _ in range(n)]

for deq in raw:
    tmp_deq=deque()
    while len(deq)>=1:
        if deq[0] not in operators:
            tmp_deq.append(float(deq.popleft()))
        else:
            b=tmp_deq.pop()
            a=tmp_deq.pop()
            operate=deq.popleft()
            deq.appendleft(cal(a,b,operate))
    print('{:.2f}'.format(tmp_deq[0]))
```

## 中缀转后缀

以下是 Shunting Yard 算法的基本步骤：

1. 初始化运算符栈和输出栈为空。
2. 从左到右遍历中缀表达式的每个符号。
  - 如果是操作数（数字），则将其添加到输出栈。
  - 如果是左括号，则将其推入运算符栈。
  - 如果是运算符：
    - 如果运算符的优先级大于运算符栈顶的运算符，或者运算符栈顶是左括号，则将当前运算符推入运算符栈。
    - 否则，将运算符栈顶的运算符弹出并添加到输出栈中，直到满足上述条件（或者运算符栈为空）。
    - 将当前运算符推入运算符栈。
  - 如果是右括号，则将运算符栈顶的运算符弹出并添加到输出栈中，直到遇到左括号。将左括号弹出但不添加到输出栈中。
3. 如果还有剩余的运算符在运算符栈中，将它们依次弹出并添加到输出栈中。
4. 输出栈中的元素就是转换后的后缀表达式。

样例输入

```
3
7+8.3
3+4.5*(7+2)
(3)*((3+4)*(2+3.5)/(4+5))
```

样例输出

```
7 8.3 +
3 4.5 7 2 + * +
3 3 4 + 2 3.5 + * 4 5 + / *
```

```
def infix_to_postfix(expression):
    def get_precedence(op):
        precedences = {'+': 1, '-': 1, '*': 2, '/': 2}
        return precedences[op] if op in precedences else 0

    def is_operator(c):
        return c in "+-*/"

    def is_number(c):
        return c.isdigit() or c == '.'

    output = []
    stack = []
    number_buffer = []

    def flush_number_buffer():
        if number_buffer:
            output.append(''.join(number_buffer))
            number_buffer.clear()

    # 主体部分
    for c in expression:
        if is_number(c):
            number_buffer.append(c)
        elif c == '(':
            flush_number_buffer()
            stack.append(c)
        elif c == ')':
            flush_number_buffer()
            while stack and stack[-1] != '(':
                output.append(stack.pop())
            stack.pop() # popping '('
        elif is_operator(c):
            flush_number_buffer()
            while stack and get_precedence(c) <= get_precedence(stack[-1]):
                output.append(stack.pop())
            stack.append(c)

    flush_number_buffer()
    while stack:
        output.append(stack.pop())

    return ' '.join(output)

# Read number of expressions
n = int(input())
# Read each expression and convert it
for _ in range(n):
    infix_expr = input()
    postfix_expr = infix_to_postfix(infix_expr)
    print(postfix_expr)
```

## 建树

```
class TreeNode:
    def __init__(self, val):
```

```
self.val=val
self.left=None
self.right=None
```

## Huffman算法

哈夫曼编码树

- 描述

构造一个具有n个外部节点的扩充二叉树，每个外部节点 $K_i$ 有一个 $W_i$ 对应，作为该外部节点的权。使得这个扩充二叉树的叶节点带权外部路径长度总和最小： $\min(W_1 * L_1 + W_2 * L_2 + W_3 * L_3 + \dots + W_n * L_n)$

$W_i$ :每个节点的权值。 $L_i$ :根节点到第 $i$ 个外部叶子节点的距离。编程计算最小外部路径长度总和。

- 输入

第一行输入一个整数n，外部节点的个数。第二行输入n个整数，代表各个外部节点的权值。  $2 \leq n \leq 100$

- 输出

输出最小外部路径长度总和。

- 样例输入

4 1 1 3 5

- 样例输出

17

```
import heapq
class HuffmanTreeNode:
    def __init__(self, weight, char=None):
        self.weight=weight
        self.char=char
        self.left=None
        self.right=None

    def __lt__(self, other):
        return self.weight<other.weight

def BuildHuffmanTree(characters):
    heap=[HuffmanTreeNode(weight, char) for char, weight in characters.items()]
    heapq.heapify(heap)
    while len(heap)>1:
        left=heapq.heappop(heap)
        right=heapq.heappop(heap)
        merged=HuffmanTreeNode(left.weight+right.weight, None)
        merged.left=left
        merged.right=right
        heapq.heappush(heap, merged)
    root=heapq.heappop(heap)
    return root

def enpaths_huffman_tree(root):
    # 字典形如(idx, weight): path
    paths={}
    def traverse(node, path):
        if node.char:
            paths[(node.char, node.weight)]=path
        else:
            traverse(node.left, path+1)
            traverse(node.right, path+1)
    traverse(root, 0)
    return paths

def min_weighted_path(paths):
    return sum(tup[1]*path for tup, path in paths.items())

n, characters=int(input()), {}
raw=list(map(int, input().split()))
for char, weight in enumerate(raw):
    characters[str(char)]=weight
root=BuildHuffmanTree(characters)
paths=enpaths_huffman_tree(root)
print(min_weighted_path(paths))
```

## 并查集

发现它，抓住它

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
```

```

self.rank = [0] * n

def find(self, x):
    if self.parent[x] != x:
        self.parent[x] = self.find(self.parent[x])
    return self.parent[x]

def union(self, x, y):
    rootX = self.find(x)
    rootY = self.find(y)
    if rootX != rootY:
        if self.rank[rootX] > self.rank[rootY]:
            self.parent[rootY] = rootX
        elif self.rank[rootX] < self.rank[rootY]:
            self.parent[rootX] = rootY
        else:
            self.parent[rootY] = rootX
            self.rank[rootX] += 1

def solve():
    n, m = map(int, input().split())
    uf = UnionFind(2 * n) # 初始化并查集，每个案件对应两个节点,一个是本身，另一个是其对立案件。
    for _ in range(m):
        operation, a, b = input().split()
        a, b = int(a) - 1, int(b) - 1
        if operation == "D":
            uf.union(a, b + n) # a与b的对立案件合并
            uf.union(a + n, b) # a的对立案件与b合并
        else: # "A"
            if uf.find(a) == uf.find(b) or uf.find(a + n) == uf.find(b + n):
                print("In the same gang.")
            elif uf.find(a) == uf.find(b + n) or uf.find(a + n) == uf.find(b):
                print("In different gangs.")
            else:
                print("Not sure yet.")

T = int(input())
for _ in range(T):
    solve()

```

食物链（三倍并查集思路简介）

类似带“敌对/捕食”关系的题用 **k倍并查集**：对每只动物开k个虚拟节点表示k种角色。如食物链(A吃B,B吃C,C吃A)：

- x 表示"x自己（同类）"， x+n 表示"x吃的"， x+2n 表示"吃x的"
- "x与y同类"  $\rightarrow$  union(x,y), union(x+n,y+n), union(x+2n,y+2n)
- "x吃y"  $\rightarrow$  union(x+n,y), union(x,y+2n), union(x+2n,y+n)
- 冲突判断：操作前若  $find(x) == find(y)$  的其他角色) 即假话

并查集求有向图极大元/极小元个数

偏序集场景：给n个元素、若干“等价关系”a~b和“小于关系”a<b（有向边a **$\rightarrow$** b），求极大元个数（没有比它更大的）和极小元个数（没有比它更小的）。

思路：

1. 用并查集把所有等价对合并到同一集合（每个集合代表一个等价类）
2. 对每条a<b的有向边：若a,b跨等价类，则a所在的根**不可能是极大**，b所在的根**不可能是极小**
3. 所有“未被标记非极大”的根  $\rightarrow$  极大元；“未被标记非极小”的根  $\rightarrow$  极小元

```

class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n + 1))

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx != ry:
            self.parent[rx] = ry

def count_extremes(n, equiv, less_than):
    # equiv: [(a,b), ...] 表示 a~b（同一等价类）
    # less_than: [(a,b), ...] 表示 a<b（有向边 a  $\rightarrow$  b）
    uf = UnionFind(n)
    for a, b in equiv:
        uf.union(a, b)

```

```

not_max = set()      # 有出边到别的类 → 非极大
not_min = set()      # 有入边来自别的类 → 非极小
for a, b in less_than:
    ra, rb = uf.find(a), uf.find(b)
    if ra != rb:
        not_max.add(ra)
        not_min.add(rb)

roots = {uf.find(i) for i in range(1, n + 1)}
num_max = len(roots - not_max)
num_min = len(roots - not_min)
return num_max, num_min

# 示例: 4个元素, 1~2 等价; 3<1, 3<4
# 等价类: {1,2}, {3}, {4}
# 跨类边: 3→{1,2}, 3→4
# 极大元: {1,2}, {4} → 2 个; 极小元: {3} → 1 个
print(count_extremes(4, [(1, 2)], [(3, 1), (3, 4)]))  # (2, 1)

```

**变形:** 若给的是一般有向图（可能含环），则先用Kosaraju/Tarjan求SCC缩点（每个SCC内的所有元素互相可达，等价），再套用上面的方法。

## Prim算法

**步骤:**

1. 起点入堆。
2. 堆顶元素出堆（排序依据是到该元素的开销），如已访问过，continue；否则标记为visited。
3. 访问该节点相邻节点，（访问开销（排序依据），相邻节点）入堆。
4. 相邻节点前驱设置为当前节点（如需）。
5. 当前节点入树

**全部精要在于：每次走出下一步的开销都是当前最小的。**

Agri-net

题目：用邻接矩阵给出图，求最小生成树路径权值和。

```

4
0 4 9 21
4 0 8 17
9 8 0 16
21 17 16 0

# 注意这一步continue很关键，因为一个节点会同时很多存在于pq中（这是由出队标记决定的）
# 如果不设计这一步continue，则会重复加路径长。

```

```

from heapq import heappop, heappush
def prim(matrix):
    ans=0
    pq,visited=[(0,0)], [False for _ in range(N)]
    while pq:
        c,cur=heappop(pq)
        if visited[cur]:continue
        visited[cur]=True
        ans+=c
        for i in range(N):
            if not visited[i] and matrix[cur][i]!=0:
                heappush(pq,(matrix[cur][i],i))
    return ans

while True:
    try:
        N=int(input())
        matrix=[list(map(int,input().split())) for _ in range(N)]
        print(prim(matrix))
    except:break

```

## Kruskal算法（能写Prim建议写Prim）

Agri-net

```

class DisJointSet:
    def __init__(self,num_vertices):
        self.parent=list(range(num_vertices))
        self.rank=[0 for _ in range(num_vertices)]

    def find(self,x):
        if self.parent[x]!=x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

```

```

def union(self,x,y):
    root_x=self.find(x)
    root_y=self.find(y)
    if root_x!=root_y:
        if self.rank[root_x]<self.rank[root_y]:
            self.parent[root_x]=root_y
        elif self.rank[root_x]>self.rank[root_y]:
            self.parent[root_y]=root_x
        else:
            self.parent[root_x]=root_y
            self.rank[root_y]+=1

# graph是邻接表
def kruskal(graph:list):
    res,edges,dsj=[],[],DisJointSet(len(graph))
    for i in range(len(graph)):
        for j in range(i+1,len(graph)):
            if graph[i][j]!=0:
                edges.append((i,j,graph[i][j]))

    for i in sorted(edges,key=lambda x:x[2]):
        u,v,weight=i
        if dsj.find(u)!=dsj.find(v):
            dsj.union(u,v)
            res.append((u,v,weight))

    return res

while True:
    try:
        n=int(input())
        graph=[list(map(int,input().split())) for _ in range(n)]
        res=kruskal(graph)
        print(sum(i[2] for i in res))
    except EOFError:break

```

## 链表优化Prim（稠密图/默认权重图）

**适用场景：**图是完全图或接近完全图，大部分边权相同（如默认为0），只有少量"特殊边"权重不同。普通堆Prim会因为 $E=V^2$ 退化为 $O(V^2\log V)$ ，链表优化能压到 $O((n+m)\log n)$ 。

**核心思想：**用双向链表维护"未访问顶点集合"。处理u时遍历链表中所有v：

- 若(u,v)是特殊边 → 推入堆
- 若(u,v)是默认边（权重最小） → 直接加入MST并从链表删除

```

# 模板：n个点完全图，默认权重0，special[u][v]=w 是特殊边
from heapq import heappush, heappop
from collections import defaultdict, deque

def prim_linked_list(n, special):
    # 双向链表，下标0和n+1是哨兵
    nxt = [i + 1 for i in range(n + 2)]
    prv = [i - 1 for i in range(n + 2)]

    def remove(v):
        nxt[prv[v]] = nxt[v]
        prv[nxt[v]] = prv[v]

    in_tree = [False] * (n + 2)
    in_tree[1] = True
    remove(1)

    bfs = deque([1])      # 0权扩展队列
    pq = []               # 特殊边堆
    ans = 0

    while bfs or pq:
        # 阶段一：0权BFS，免费扩张
        while bfs:
            u = bfs.popleft()
            v = nxt[0]
            while v <= n:
                v_next = nxt[v]
                if v not in special[u]:      # 默认0权边
                    in_tree[v] = True
                    remove(v)
                    bfs.append(v)
                else:                        # 特殊边压堆
                    heappush(pq, (special[u][v], v))
                v = v_next

        # 阶段二：堆中取最小特殊边连接一个新块

```



```

while pq:
    w, v = heappop(pq)
    if in_tree[v]:
        continue
    in_tree[v] = True
    remove(v)
    ans += w
    bfs.append(v)
    break

return ans

while stack:
    u = stack.pop()
    # 临时移除 u 的所有原图邻居
    temp = []
    for v in adj[u]:
        if comp[v] == 0: # 尚未被分配分量，仍在链表中
            # 临时删除 v
            nxt[pre[v]] = nxt[v]
            pre[nxt[v]] = pre[v]
            temp.append(v)

    # 此时链表中剩下的全是补图邻居，全部加入当前分量
    v = nxt[0]
    while v != 0:
        nxt_v = nxt[v]
        comp[v] = cid
        remove(v) # 永久删除
        stack.append(v)
        v = nxt_v

    # 恢复临时删除的邻居
    for v in temp:
        insert_front(v)

```

**复杂度分析：**每个顶点最多从链表删除1次（ $O(n)$ ）；每条特殊边最多被两端各扫一次入堆（ $O(m)$ ）；堆操作 $O(\log n)$ 。总 $O((n+m)\log n)$ 。

## T30313: 0-W 最小生成树（完全图，少量正权边）

$n$ 个点完全图， $m$ 条边权为 $w>0$ ，其余边权为0。求MST权值和。 $n\leq 1e5, m\leq 2e5$ 。

**样例**

| 输入     | 输出 |
|--------|----|
| 6 11   | 15 |
| 1 3 10 |    |
| 1 4 10 |    |
| 1 5 10 |    |
| 1 6 10 |    |
| 2 3 10 |    |
| 2 4 10 |    |
| 2 5 10 |    |
| 2 6 10 |    |
| 3 4 5  |    |
| 3 5 6  |    |
| 3 6 7  |    |

**思路：**0权边把图分成若干“块”（0-边连通分量），块内连边免费；块间只能用特殊边连接，对块缩点后跑MST即可。

**解法一：链表优化Prim**

```

import sys
from heapq import heappush, heappop
from collections import defaultdict, deque

def main():
    data = sys.stdin.buffer.read().split()
    p = 0
    n, m = int(data[p]), int(data[p+1]); p += 2
    special = defaultdict(dict)
    for _ in range(m):
        u, v, w = int(data[p]), int(data[p+1]), int(data[p+2]); p += 3
        special[u][v] = w
        special[v][u] = w

    nxt = [i + 1 for i in range(n + 2)]
    prv = [i - 1 for i in range(n + 2)]

    def remove(v):
        nxt[prv[v]] = nxt[v]
        prv[nxt[v]] = prv[v]

    in_tree = [False] * (n + 2)

```

```

in_tree[1] = True
remove(1)

bfs = deque([1])
pq = []
ans = 0

while bfs or pq:
    while bfs:
        u = bfs.popleft()
        v = nxt[0]
        while v <= n:
            v_next = nxt[v]
            if v not in special[u]:
                in_tree[v] = True
                remove(v)
                bfs.append(v)
            else:
                heappush(pq, (special[u][v], v))
            v = v_next

    while pq:
        w, v = heappop(pq)
        if in_tree[v]:
            continue
        in_tree[v] = True
        remove(v)
        ans += w
        bfs.append(v)
        break

print(ans)

main()

```

## 解法二：并查集缩块 + Kruskal

更直观：先用链表BFS找出所有0权连通块，然后按特殊边权排序，用并查集合并块。

```

import sys
from collections import defaultdict

def main():
    data = sys.stdin.buffer.read().split()
    p = 0
    n, m = int(data[p]), int(data[p+1]); p += 2
    special = defaultdict(set)
    edges = []
    for _ in range(m):
        u, v, w = int(data[p]), int(data[p+1]), int(data[p+2]); p += 3
        special[u].add(v)
        special[v].add(u)
        edges.append((w, u, v))

    # 第一步：链表BFS找0权连通块
    nxt = [i + 1 for i in range(n + 2)]
    prv = [i - 1 for i in range(n + 2)]

    def remove(v):
        nxt[prv[v]] = nxt[v]
        prv[nxt[v]] = prv[v]

    comp = [0] * (n + 1)      # comp[v]=块号
    cid = 0
    for start in range(1, n + 1):
        if comp[start]:
            continue
        cid += 1
        comp[start] = cid
        remove(start)
        stack = [start]
        while stack:
            u = stack.pop()
            v = nxt[0]
            while v <= n:
                v_next = nxt[v]
                if v not in special[u]:      # 0权边
                    comp[v] = cid
                    remove(v)
                    stack.append(v)
                v = v_next

```

```
# 第二步：在块上跑Kruskal
parent = list(range(cid + 1))
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]
        x = parent[x]
    return x

edges.sort()
ans = 0
used = 0
for w, u, v in edges:
    cu, cv = find(comp[u]), find(comp[v])
    if cu != cv:
        parent[cu] = cv
        ans += w
        used += 1
        if used == cid - 1:      # 已连通所有块，提前结束
            break

print(ans)

main()
```

两种解法对比

| 解法         | 复杂度                                 | 优劣               |
|------------|-------------------------------------|------------------|
| 链表Prim     | $O((n+m)\log n)$                    | 一次遍历完成，代码紧凑      |
| 缩块+Kruskal | $O(n+m\cdot\alpha(n)+m\cdot\log m)$ | 思路更清晰，分两步：先建块再连块 |

样例验证：0权边把 {1,2}, {4,5,6} 各连成块，{3}单独成块。共3块需2条正权边连接。最小为  $3-4:5 + 1-3:10 = \mathbf{15}$ 。✓

Kahn算法

Kahn算法的基本思想是通过不断地移除图中的入度为0的顶点，并将其添加到拓扑排序的结果中，直到图中所有的顶点都被移除。具体步骤如下：

1. 初始化一个队列，用于存储当前入度为0的顶点。
2. 遍历图中的所有顶点，计算每个顶点的入度，并将入度为0的顶点加入到队列中。
3. 不断地从队列中弹出顶点，并将其加入到拓扑排序的结果中。同时，遍历该顶点的邻居，并将其入度减1。如果某个邻居的入度减为0，则将其加入到队列中。
4. 重复步骤3，直到队列为空。

**Kahn算法的时间复杂度为 $O(V + E)$** ，其中**V**是顶点数，**E**是边数。它是一种简单而高效的拓扑排序算法，在有向无环图（DAG）中广泛应用。

拓扑排序

题目：给出一个图的结构，输出其拓扑排序序列，要求在同等条件下，编号小的顶点在前。

题解中graph是邻接表，形如graph[1]=[2,3,4]，由于本题要求顺序，因此不用队列而用优先队列。

```
from collections import defaultdict
from heapq import heappush,heappop
def Kahn(graph):
    q,ans=[],[]
    in_degree=defaultdict(int)
    for lst in graph.values():
        for vert in lst:
            in_degree[vert]+=1

    for vert in graph.keys():
        if vert not in in_degree or in_degree[vert]==0:
            heappush(q,vert)

    while q:
        vertex=heappop(q)
        ans.append('v'+str(vertex))
        for neighbor in graph[vertex]:
            in_degree[neighbor]-=1
            if in_degree[neighbor]==0:
                heappush(q,neighbor)

    return ans

v,a=map(int,input().split())
graph={}
for _ in range(a):
    f,t=map(int,input().split())
    if f not in graph:graph[f]=[]
    if t not in graph:graph[t]=[]
    graph[f].append(t)

for i in range(1,v+1):
```

```

    if i not in graph:graph[i]=[]

res=Kahn(graph)
print(*res)

```

## DFS求拓扑序列（带环检测）

思路：对图DFS，计算每个顶点的"结束时间"（Finish Time），把顶点按结束时间**递减**排列即得拓扑序。

三色标记：白(0)未访问，灰(1)正在访问（在递归栈中），黑(2)访问完毕。若DFS中遇到灰色节点说明指向了祖先，存在环。

**算法原理**：对任意有向边  $u \rightarrow v$

- 若v未访问：DFS u 时会递归并先访问完v，所以v比u先结束
- 若v已结束：v显然先于u结束
- 所以总有  $finish[v] < finish[u]$ ，按结束时间降序即拓扑序

```

def dfs_topo_sort(graph):
    # 0=白未访问, 1=灰正在访问, 2=黑访问完毕
    visited = {v: 0 for v in graph}
    finish_order = []

    def dfs_visit(u):
        visited[u] = 1
        for v in graph.get(u, []):
            if visited[v] == 0:
                if not dfs_visit(v):
                    return False
            elif visited[v] == 1:
                return False          # 指向祖先节点, 发现环
        visited[u] = 2
        finish_order.append(u)        # 节点结束时追加到结果列表
        return True

    for u in graph:                  # 处理非连通图/森林
        if visited[u] == 0:
            if not dfs_visit(u):
                raise ValueError("图中存在环, 无法进行拓扑排序")

    return finish_order[::-1]        # 结束时间递减序即拓扑序

# 示例（邻接表）：做煎饼的工序依赖
pancake_graph = {
    "3/4 cup milk": ["1 cup mix"],
    "1 egg": ["1 cup mix"],
    "1 tsp oil": ["1 cup mix"],
    "1 cup mix": ["pour 1/4 cup"],
    "heat griddle": ["pour 1/4 cup"],
    "pour 1/4 cup": ["turn when bubbly"],
    "turn when bubbly": ["eat"],
    "eat": []
}
print(" -> ".join(dfs_topo_sort(pancake_graph)))
# heat griddle -> 1 tsp oil -> 1 egg -> 3/4 cup milk -> 1 cup mix
# -> pour 1/4 cup -> turn when bubbly -> eat

```

**Kahn vs DFS 选择**：要按编号顺序输出选Kahn（堆+入度），只需任一拓扑序或同时要判环选DFS。

## Hierholzer算法（欧拉路径/欧拉回路）

**欧拉路径**：经过每条边恰好一次的路径。**欧拉回路**：起终点相同的欧拉路径。

**存在条件**：

- **有向图回路**：每个点入度=出度，且所有有边的点连通
- **有向图路径**：恰有一点出度-入度=1（起点）、一点入度-出度=1（终点），其余入度=出度
- **无向图**：所有度数为偶数→回路；恰有2个奇度点→路径（起终点必为奇度点）

**算法思想**：从合法起点DFS，每走过一条边就删除（避免重走），无路可走时把当前点压入答案栈；最后反转栈即得欧拉路径。每条边访问一次， $O(V+E)$ 。

**最小字典序欧拉路径**：邻接表用最小堆维护，DFS每次取当前最小的邻接点。

```

import sys
from collections import defaultdict
import heapq
sys.setrecursionlimit(1 << 25)

def hierholzer_min_lex(n, edges):
    """有向图最小字典序欧拉路径。返回顶点列表，不存在返回None."""
    graph = defaultdict(list)
    indeg = [0]*(n+1)
    outdeg = [0]*(n+1)

```

```

for u, v in edges:
    graph[u].append(v)
    outdeg[u] += 1
    indeg[v] += 1

for u in graph:
    heapq.heapify(graph[u])          # 最小堆保证字典序最小

# 检查欧拉路径存在条件
start = -1
end = -1
for i in range(1, n+1):
    diff = outdeg[i] - indeg[i]
    if diff == 1:
        if start != -1: return None
        start = i
    elif diff == -1:
        if end != -1: return None
        end = i
    elif diff != 0:
        return None
if start == -1 and end == -1:
    # 欧拉回路：从有边的最小顶点开始
    start = min((v for v in range(1, n+1) if outdeg[v] > 0), default=1)
elif not (start != -1 and end != -1):
    return None

path = []
def dfs(v):
    while graph[v]:
        u = heapq.heappop(graph[v])
        dfs(u)
    path.append(v)

dfs(start)
path.reverse()
return path if len(path) == len(edges)+1 else None

# 示例
edges = [(1,2),(2,3),(3,1),(1,4),(4,1)]
print(hierholzer_min_lex(4, edges))    # [1, 2, 3, 1, 4, 1]

```

迭代版（防递归栈溢出）：

```

def hierholzer_iter(start, graph):
    # graph[v] 已 heapify
    stack, path = [start], []
    while stack:
        v = stack[-1]
        if graph[v]:
            stack.append(heapq.heappop(graph[v]))
        else:
            path.append(stack.pop())
    return path[::-1]

```

## 关键路径（AOE网）

定义：AOE网（Activity On Edge）是带权DAG，**边代表活动、权值为耗时，顶点代表事件**。关键路径 = 从源点到汇点的**最长路径**，关键路径上的活动没有任何“机动时间”（slack=0），延迟任一关键活动会拖延整个工程。

两阶段算法：

1. **正向拓扑序**求  $ve[v]$ （最早发生时间）： $ve[v] = \max(ve[u] + w(u,v))$  over前驱 $u$ 。源点 $ve=0$ 。
2. **逆拓扑序**求  $vl[v]$ （最晚发生时间）： $vl[u] = \min(vl[v] - w(u,v))$  over后继 $v$ 。汇点 $vl=ve$ 。
3. 对边 $(u,v,w)$ ：若  $ve[u] == vl[v] - w$ ，则它是关键活动。所有关键活动构成关键路径的子图（可能有多条并行的关键路径）。

```

import sys
from collections import defaultdict, deque

def critical_path(n, edges):
    # edges = [(u, v, w), ...]
    G = defaultdict(list)
    in_deg = [0] * n
    for u, v, w in edges:
        G[u].append((v, w))
        in_deg[v] += 1

    # 1. 拓扑序求 ve
    deg = in_deg[:]
    q = deque(i for i in range(n) if deg[i] == 0)
    ve = [0] * n

```

```

order = []
while q:
    u = q.popleft()
    order.append(u)
    for v, w in G[u]:
        ve[v] = max(ve[v], ve[u] + w)
        deg[v] -= 1
        if deg[v] == 0:
            q.append(v)
if len(order) != n:
    return -1, [] # 有环

total = max(ve)

# 2. 逆拓扑序求 vl
vl = [total] * n
for u in reversed(order):
    for v, w in G[u]:
        vl[u] = min(vl[u], vl[v] - w)

# 3. 找关键活动: ve[u] + w == vl[v]
critical = []
for u, v, w in edges:
    if ve[u] + w == vl[v]:
        critical.append((u, v, w))
return total, critical

# 测试: 6点8边
edges = [(0,1,3),(0,2,2),(1,3,2),(1,4,3),(2,3,4),(2,5,1),(3,5,2),(4,5,1)]
total, critical = critical_path(6, edges)
print("工期:", total) # 8
print("关键活动:", critical) # [(0,2,2),(2,3,4),(3,5,2)] → 路径0→2→3→5

```

打印所有关键路径（用关键活动建子图DFS）：

```

def print_paths(start, crit_adj, path=None):
    if path is None: path = []
    path.append(start)
    if not crit_adj[start]:
        print(" -> ".join(map(str, path)))
    else:
        for v in sorted(crit_adj[start]):
            print_paths(v, crit_adj, path[:])
    path.pop()

crit_adj = defaultdict(list)
for u, v, w in critical:
    crit_adj[u].append(v)
# 对所有入度为0的点调用 print_paths

```

**多条并行关键路径：**若一个节点有多条出去的关键活动，DFS会枚举出所有路径。

## Dijkstra算法

道路（更推荐第二种剪枝写法）

N个以 1 ... N 标号的城市通过单向的道路相连。每条道路包含两个参数：道路的长度和需要为该路付的通行费（以金币的数目来表示）。Bob从1到N。他希望能够尽可能快的到那，但是他囊中羞涩。我们希望能够帮助Bob找到从1到N最短的路径，前提是他能够付的起通行费。输出结果应该只包括一行，即从城市1到城市N所需的最小的路径长度（花费不能超过K个金币）。如果这样的路径不存在，结果应该输出-1。

S：起点；D：终点；L：道路长；T：通行费。

```

from heapq import heappop, heappush
from collections import defaultdict
K, N, R = int(input()), int(input()), int(input())
graph = defaultdict(list)
for i in range(R):
    S, D, L, T = map(int, input().split())
    graph[S].append((D, L, T))
def Dijkstra(graph):
    global K, N, R
    q, ans = [], []
    heappush(q, (0, 0, 1, 0))
    while q:
        l, cost, cur, step = heappop(q)
        if cur == N: return l
        for next, nl, nc in graph[cur]:
            # 剪枝: 如果步数不少于N: 意味着一定走了回头路, 减掉。
            if cost + nc <= K and step + 1 < N:
                heappush(q, (l + nl, cost + nc, next, step + 1))

```

```

    return -1
print(Dijkstra(graph))

```

```

from heapq import heappop,heappush
from collections import defaultdict
K,N,R=int(input()),int(input()),int(input())
graph=defaultdict(list)
for i in range(R):
    S,D,L,T=map(int,input().split())
    graph[S].append((D,L,T))

def Dijkstra(graph):
    global K,N,R
    q,ans=[],[]
    min_cost={i:float('inf') for i in range(1,N+1)}
    heappush(q,(0,0,1))
    while q:
        l,cost,cur=heappop(q)
        min_cost[cur]=min(min_cost[cur],cost)
        if cur==N:return l
        for next,nl,nc in graph[cur]:
            # 剪枝1: 只有花费小于等于K才能入堆。
            # 剪枝2: 只有到达下一个节点的花费比上次更小时才能入堆（否则路程长花费大，无意义）。
            if cost+nc<=K and nc+cost<min_cost[next]:
                heappush(q,(l+nl,cost+nc,next))

    return -1
print(Dijkstra(graph))

```

## Dijkstra通用模板（推荐背诵版）

```

import heapq
def dijkstra(n, adj, start):
    # adj[u] = [(v, weight), ...]
    dist = [float('inf')] * n
    dist[start] = 0
    pq = [(0, start)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]: continue # 出堆标记: 陈旧记录跳过
        for v, w in adj[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))
    return dist

```

时间复杂度 $O((V+E)\log V)$ 。仅适用于非负权边（有负边时贪心策略失效，可能死循环）。

## Bellman-Ford算法（支持负权边）

核心思想是动态规划而非贪心：每轮把所有边都松弛一遍，重复 $V-1$ 轮。

逻辑证明： $n$ 个点的最短路径最多 $V-1$ 条边， $V-1$ 次松弛保证信息能从源点逐层传递到任何点。第 $V$ 次还能更新说明存在负环。

```

def bellman_ford(n, edges, start):
    # edges = [(u, v, w), ...]
    dist = [float('inf')] * n
    dist[start] = 0
    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    # 第V次还能松弛 -> 存在负环
    for u, v, w in edges:
        if dist[u] != float('inf') and dist[u] + w < dist[v]:
            return "存在负权环"
    return dist

edges = [(0,1,5),(0,2,4),(1,3,3),(2,1,6),(3,2,-2)]
print(bellman_ford(4, edges, 0)) # [0, 5, 4, 8]

```

时间复杂度 $O(V \cdot E)$ 。

### 典型应用简介：

- 限制最多 $K$ 次中转的最短路（LC787）：中转 $K$ 次=最多 $K+1$ 条边，只做 $K+1$ 轮松弛。每轮基于上轮副本 `prev = dist[:]` 避免本轮串扰

## 应用：货币兑换套利（OJ01860）

题意： $N$ 种货币、 $M$ 个兑换点，每个兑换点给出双向汇率和手续费。Nick初始持有 $V$ 单位货币 $S$ ，问能否通过若干次兑换让 $S$ 的数量增加（最后仍持有 $S$ ）。

**输入：**第一行  $N\ M\ S\ V$ ；接下来M行  $A\ B\ Rab\ Cab\ Rba\ Cba$ ，表示 $A \leftrightarrow B$ 兑换的汇率和手续费。

**样例：** 3 2 1 20.0 + 两个兑换点  $\rightarrow$  YES

**思路：**把货币当顶点，每个兑换点产生两条有向边（ $A \rightarrow B$  和  $B \rightarrow A$ ），松弛操作改为**非线性**：

$$= (( ))$$

注意这里求"最大金额"用 max 而不是 min（最短路求最小，套利求最大）。**判定条件：**做N轮松弛后若能更新（或 $best[S] > V$ ），说明存在**正增益环**，套利可行。

**思想扩展：**BF/SPFA的核心是"反复松弛+检测V轮后还能否更新"，这套框架可推广为：

- 求**最短/最长路径**（min/max 取决于问题）
- 检测**负权环**（差分约束、套利不可能等场景）
- 检测**正权环**（这道题：从某点回到自身权重能否变得更优）—— 本质都是"点到自身"是否能改进

**BF版本（完整代码）**

```
import sys

def main():
    data = sys.stdin.read().split()
    p = 0
    N, M = int(data[p]), int(data[p+1])
    S, V = int(data[p+2]), float(data[p+3])
    p += 4
    edges = []
    for _ in range(M):
        A, B = int(data[p]), int(data[p+1])
        Rab, Cab = float(data[p+2]), float(data[p+3])
        Rba, Cba = float(data[p+4]), float(data[p+5])
        p += 6
        edges.append((A, B, Rab, Cab))
        edges.append((B, A, Rba, Cba))

    best = [0.0] * (N + 1)
    best[S] = V

    # 最多N-1轮松弛：若第N轮仍能松弛 -> 存在正增益环
    for it in range(N):
        updated = False
        for u, v, rate, fee in edges:
            if best[u] > fee:
                x = (best[u] - fee) * rate
                if x > best[v] + 1e-12:
                    best[v] = x
                    updated = True
        if not updated:
            break

    print("YES" if best[S] > V else "NO")

main()
```

**SPFA版本（同样可行，常数更小）**

SPFA本质上是BF的队列优化：只把"刚更新过"的点加入队列，节省冗余松弛。判定套利的方法是：某点入队次数  $\geq N$  时说明存在正增益环。

```
import sys
from collections import deque

def main():
    data = sys.stdin.read().split()
    p = 0
    N, M = int(data[p]), int(data[p+1])
    S, V = int(data[p+2]), float(data[p+3])
    p += 4
    adj = [[] for _ in range(N + 1)]
    for _ in range(M):
        A, B = int(data[p]), int(data[p+1])
        Rab, Cab = float(data[p+2]), float(data[p+3])
        Rba, Cba = float(data[p+4]), float(data[p+5])
        p += 6
        adj[A].append((B, Rab, Cab))
        adj[B].append((A, Rba, Cba))

    best = [0.0] * (N + 1)
    best[S] = V
    in_queue = [False] * (N + 1)
    count = [0] * (N + 1)
    q = deque([S])
    in_queue[S] = True
```



```

while q:
    u = q.popleft()
    in_queue[u] = False
    for v, rate, fee in adj[u]:
        if best[u] <= fee: continue
        x = (best[u] - fee) * rate
        if x > best[v] + 1e-12:
            best[v] = x
            if v == S and best[S] > V:          # 可提前返回
                print("YES"); return
            if not in_queue[v]:
                q.append(v); in_queue[v] = True
                count[v] += 1
            if count[v] > N:                    # 入队>N次 → 正环
                print("YES"); return

print("YES" if best[S] > V else "NO")

main()

```

**两种方法的等价思想：**本题本质是问“从S出发能否通过一条路径回到S，且总收益(乘积)>1”。这等价于判定一个点到自身的最优路径是否更优——

- 求最短路：检测负环（点回自身能变得更短，即图中存在权值<0的环）
- 求最长路/最大收益：检测正环（点回自身能变得更优，即图中存在权值>0的环或收益>1的环）

BF用V轮固定迭代后看是否还有更新，SPFA用入队次数计数，两者都是同一思想的不同实现。

## SPFA（Bellman-Ford的队列优化）

核心：只有dist[u]被更新过，u的邻居才可能变短。所以维护一个“待松弛”队列，避免每轮盲目扫描所有边。

**负环判定：**某点入队次数  $\geq V$  时存在负环（包含V条边的路径必有重复点）。

```

from collections import deque

def spfa(graph, V, source):
    # graph[u] = [(v, w), ...]
    dist = [float('inf')] * V
    dist[source] = 0
    queue = deque([source])
    in_queue = [False] * V
    in_queue[source] = True
    count = [0] * V          # 入队次数（用于判负环）
    count[source] = 1

    while queue:
        u = queue.popleft()
        in_queue[u] = False
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                if not in_queue[v]:
                    count[v] += 1
                    if count[v] >= V:
                        return "存在负权环"
                # SLF优化：距离更小的插队首
                if queue and dist[v] < dist[queue[0]]:
                    queue.appendleft(v)
                else:
                    queue.append(v)
            in_queue[v] = True

    return dist

# 用法：把edges转邻接表
edges = [(0,1,5),(0,2,4),(1,3,3),(2,1,6),(3,2,-2)]
V = 4
graph = [[] for _ in range(V)]
for u, v, w in edges:
    graph[u].append((v, w))
print(spfa(graph, V, 0))  # [0, 5, 4, 8]

```

**注意事项：**平均 $O(k \cdot E)$  ( $k \geq 2$ )，最坏 $O(V \cdot E)$ 。无负权边时优先用Dijkstra ( $O((V+E)\log V)$ 更稳定)。

## Floyd-Warshall算法（多源最短路径）

求**所有点对**之间的最短路径。基于动态规划：dist[k][i][j] 表示只用前k个点作中转时i到j的最短路。

转移：dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])，可压缩成二维。

**注意：**中间点k必须在最外层循环！

```
def floyd_warshall(V, graph):
    # graph是邻接矩阵，无边为inf，自身到自身为0
    dist = [row[:] for row in graph]
    for k in range(V):
        for i in range(V):
            for j in range(V):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    return dist

INF = float('inf')
graph = [
    [0, 5, INF, 10],
    [INF, 0, 3, INF],
    [INF, INF, 0, 1],
    [INF, INF, INF, 0]
]
for row in floyd_warshall(4, graph):
    print(row)
```

时间复杂度 $O(V^3)$ ，适合 $V \leq 500$ 左右的稠密图、所有点对查询、含负权边。

**负环检测：**算法结束后若  $dist[i][i] < 0$ ，说明在某个负权环上。

```
for i in range(V):
    if dist[i][i] < 0:
        print("存在负环"); break
```

**应用：带路径还原（兔子与樱花 OJ05443）**

用  $next\_node[i][j]$  记录 $i$ 到的下一个节点，更新时同步更新。

```
def floyd_with_path(V, graph):
    dist = [row[:] for row in graph]
    nxt = [[j if dist[i][j] != INF else -1 for j in range(V)] for i in range(V)]
    for k in range(V):
        for i in range(V):
            for j in range(V):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
                    nxt[i][j] = nxt[i][k] # 关键：i到j的下一步=i到k的下一步

    return dist, nxt

def reconstruct(nxt, u, v):
    if nxt[u][v] == -1: return None
    path = [u]
    while u != v:
        u = nxt[u][v]
        path.append(u)
    return path
```

**四种最短路算法对比**

| 算法             | 时间复杂度                                | 负权边 | 负环检测                    | 适用场景                  |
|----------------|--------------------------------------|-----|-------------------------|-----------------------|
| Dijkstra       | $O((V+E)\log V)$                     | ×   | ×                       | 单源、非负权（首选）            |
| Bellman-Ford   | $O(V \cdot E)$                       | ✓   | ✓                       | 单源、有负权、限制边数           |
| SPFA           | 平均 $O(k \cdot E)$ ，最坏 $O(V \cdot E)$ | ✓   | ✓                       | 单源、稀疏图、有负权            |
| Floyd-Warshall | $O(V^3)$                             | ✓   | ✓ (查 $dist[i][i] < 0$ ) | 多源、 $V \leq 500$ 、有负权 |

## 最小斯坦纳树（状压DP + Dijkstra）

**问题：**给无向带权图 $G$ 和 $k$ 个**关键点**（ $k$ 通常 $\leq 10$ ），求一棵边权和最小的子树，包含所有关键点。子树允许经过非关键点作"中转"（斯坦纳点）。

**意义：**MST只在给定点集内连边，斯坦纳树允许借助其他节点缩短总长度。NP-hard，但 $k$ 小时可用  $O(n \cdot 3^k + (n+m) \cdot 2^k \cdot \log n)$  解出。

**状态：** $dp[mask][i]$  = 以 $i$ 为"根"（当前所在节点）、已连通关键点集合为 $mask$ 的最小代价。

**转移：**

- 阶段A（**同点合并子集**）：枚举 $mask$ 的非空真子集 $sub$ ， $dp[mask][i] = \min(dp[mask][i], dp[sub][i] + dp[mask \wedge sub][i])$
- 阶段B（**Dijkstra扩展**）：对每个固定的 $mask$ ，把所有 $dp[mask][\cdot]$ 作为多源初始距离跑Dijkstra，沿边松弛更新整张图

**重要技巧：** $sub = (sub - 1) \& mask$  是**枚举mask所有非空子集**的经典写法。每次将 $sub-1$ 再与 $mask$ 取与，能在  $O(3^n)$  总时间内遍历所有  $(mask, sub)$  对（其中 $sub \subseteq mask$ ），而不是  $O(4^n)$ 。证明：每位有3种状态——在 $mask$ 外/在 $mask$ 和 $sub$ 中/在 $mask$ 中但不在 $sub$ 中。状压DP中频繁使用。

```
import heapq, sys
from itertools import islice
```

```
def steiner_tree(n, edges, terminals):
    # n: 节点数(1..n), edges: [(u,v,w),...], terminals: 关键点列表
    k = len(terminals)
    adj = [[] for _ in range(n + 1)]
    for u, v, w in edges:
        adj[u].append((v, w))
        adj[v].append((u, w))

    INF = float('inf')
    # dp[mask][i]: 连通关键点集合mask、根在i的最小代价
    dp = [[INF] * (n + 1) for _ in range(1 << k)]
    for i, t in enumerate(terminals):
        dp[1 << i][t] = 0  # 单个关键点 自连代价0

    for mask in range(1, 1 << k):
        # 阶段A: 枚举子集 sub ⊆ mask 合并
        sub = (mask - 1) & mask  # 子集枚举模板
        while sub > 0:
            for i in range(1, n + 1):
                if dp[sub][i] + dp[mask ^ sub][i] < dp[mask][i]:
                    dp[mask][i] = dp[sub][i] + dp[mask ^ sub][i]
            sub = (sub - 1) & mask  # 关键: 移到mask的下一个子集

        # 阶段B: 对固定mask跑Dijkstra
        pq = [(dp[mask][i], i) for i in range(1, n + 1) if dp[mask][i] < INF]
        heapq.heapify(pq)
        while pq:
            d, u = heapq.heappop(pq)
            if d > dp[mask][u]: continue
            for v, w in adj[u]:
                if d + w < dp[mask][v]:
                    dp[mask][v] = d + w
                    heapq.heappush(pq, (dp[mask][v], v))

    return min(dp[(1 << k) - 1][1:])

# 样例: 7点7边, 关键点{2,4,7,5}, 答案11
edges = [(1,2,3),(2,3,2),(4,3,9),(2,6,2),(4,5,3),(6,5,2),(7,6,4)]
print(steiner_tree(7, edges, [2, 4, 7, 5]))  # 11
```

子集枚举的几种常见形式:

```
# 1. 枚举mask的所有非空真子集
sub = (mask - 1) & mask
while sub > 0:
    # 用 sub
    sub = (sub - 1) & mask

# 2. 枚举包括空集和自身在内的所有子集
sub = mask
while True:
    # 用 sub
    if sub == 0: break
    sub = (sub - 1) & mask

# 3. 枚举全集的所有子集对(S, T)使S∩T=∅、S∪T=全集 → 即sub和mask^sub
# 直接用上面的写法配合 mask^sub 即可
```

## Kosaraju算法

```
def dfs1(graph, node, visited, stack):
    visited[node] = True
    for neighbor in graph[node]:
        if not visited[neighbor]:
            dfs1(graph, neighbor, visited, stack)
    stack.append(node)

def dfs2(graph, node, visited, component):
    visited[node] = True
    component.append(node)
    for neighbor in graph[node]:
        if not visited[neighbor]:
            dfs2(graph, neighbor, visited, component)

def kosaraju(graph):
    # Step 1: Perform first DFS to get finishing times
    stack = []
    visited = [False] * len(graph)
    for node in range(len(graph)):
        if not visited[node]:
            dfs1(graph, node, visited, stack)

    # Step 2: Perform second DFS on reversed graph to find components
```

```

# Step 2: Transpose the graph
transposed_graph = [[] for _ in range(len(graph))]
for node in range(len(graph)):
    for neighbor in graph[node]:
        transposed_graph[neighbor].append(node)

# Step 3: Perform second DFS on the transposed graph to find SCCs
visited = [False] * len(graph)
sccs = []
while stack:
    node = stack.pop()
    if not visited[node]:
        scc = []
        dfs2(transposed_graph, node, visited, scc)
        sccs.append(scc)

return sccs

# Example
graph = [[1], [2, 4], [3, 5], [0, 6], [5], [4], [7], [5, 6]]
sccs = kosaraju(graph)
print("Strongly Connected Components:")
for scc in sccs:
    print(scc)

"""
Strongly Connected Components:
[0, 3, 2, 1]
[6, 7]
[5, 4]
"""

```

## Tarjan算法（强连通分量SCC）

**思想：**一次DFS即可求出所有SCC。给每个节点维护两个值：

- $idx[v]$ ：首次访问 $v$ 的时间戳
- $low[v]$ ： $v$ 能到达的最早时间戳（含自身及其后代的回边目标）

当  $low[v] == idx[v]$  时 $v$ 是SCC的根，从栈中弹出元素直到 $v$ ，这些元素组成一个SCC。  $O(V+E)$ 。

```

import sys
sys.setrecursionlimit(1 << 25)

def tarjan_scc(n, edges):
    graph = [[] for _ in range(n+1)]
    for u, v in edges:
        graph[u].append(v)

    index = 0
    idx = [0]*(n+1)
    low = [0]*(n+1)
    on_stack = [False]*(n+1)
    stack = []
    sccs = []

    def dfs(v):
        nonlocal index
        index += 1
        idx[v] = low[v] = index
        stack.append(v)
        on_stack[v] = True
        for w in graph[v]:
            if idx[w] == 0:          # 未访问 → 树边
                dfs(w)
                low[v] = min(low[v], low[w])
            elif on_stack[w]:        # 在栈中 → 回边
                low[v] = min(low[v], idx[w])
        if low[v] == idx[v]:         # v是SCC的根
            comp = []
            while True:
                w = stack.pop()
                on_stack[w] = False
                comp.append(w)
                if w == v:
                    break
            sccs.append(comp)

    for v in range(1, n+1):
        if idx[v] == 0:

```

```
    dfs(v)
    return sccs
```

**Tarjan vs Kosaraju**：两者均 $O(V+E)$ 。Tarjan只跑一次DFS常数小、不用建反图；Kosaraju要建反图扫两次但代码更直观。Tarjan返回的SCC顺序天然是在缩点后DAG的逆拓扑序，缩点后处理很方便。

## 无向图判断连通和成环

判断无向图是否连通有无回路

```
from collections import defaultdict, deque
# graph是邻接表{1:[2,3,4]}
def is_connected(graph, n):
    dq = deque()
    dq.append(0)
    visited = set()
    visited.add(0)
    while dq:
        cur_vert = dq.popleft()
        for next_vert in graph[cur_vert]:
            if next_vert not in visited:
                dq.append(next_vert)
                visited.add(next_vert)
    return len(visited) == n

def is_loop(graph):
    global_visited = set()
    for vertex in graph:
        if vertex not in global_visited:
            # 以下是一个BFS函数。
            local_visited = {}
            dq = deque()
            dq.append((vertex, 0))
            local_visited[vertex] = 0
            global_visited.add(vertex)
            while dq:
                cur_vert, steps = dq.popleft()
                for next_vert in graph[cur_vert]:
                    if next_vert in local_visited:
                        if local_visited[next_vert] >= steps:
                            return True
                    else:
                        dq.append((next_vert, steps+1))
                        local_visited[next_vert] = steps+1
                        global_visited.add(next_vert)
    return False

n, m = map(int, input().split())
graph = defaultdict(list)
for _ in range(m):
    a, b = map(int, input().split())
    graph[a].append(b)
    graph[b].append(a)
print('connected:yes' if is_connected(graph, n) else 'connected:no')
print('loop:yes' if is_loop(graph) else 'loop:no')
```

### 方法二：并查集判环（推荐，最快）

原理：初始每点独立成集合。逐条扫描边，若边的两 endpoint 已经在同一集合中，加入这条边就会成环。适合稠密图、边数多的情形。时间复杂度近似 $O(\alpha(n) \cdot E)$ ， $\alpha$ 是阿克曼反函数（接近常数）。

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x]) # 路径压缩
        return self.parent[x]

    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False # 已同集合，加这条边成环
        self.parent[ry] = rx
        return True

def has_cycle_uf(n, edges):
    uf = UnionFind(n)
    for u, v in edges:
        if not uf.union(u, v):
```

```
        return True
    return False

# 0-1-2-0 构成环
edges = [(0, 1), (1, 2), (2, 0)]
print(has_cycle_uf(3, edges))    # True
```

方法三：DFS+parent指针判环（最直观）

原理：DFS遍历时记录当前节点的父亲。若遇到一个已访问的邻居且它不是当前节点的父亲，说明从另一条路绕回来了，存在环。  
区分关键：

- 邻居 == parent：这是刚走过来的路，正常
- 邻居 ≠ parent 且已 visited：从另一条路回到老地方，成环

```
import sys
sys.setrecursionlimit(100000)

def has_cycle_dfs(graph, n):
    visited = [False] * n

    def dfs(u, parent):
        visited[u] = True
        for v in graph[u]:
            if not visited[v]:
                if dfs(v, u):
                    return True
            elif v != parent:
                return True
        return False

    for u in range(n):          # 处理非连通图/森林
        if not visited[u]:
            if dfs(u, -1):
                return True
    return False

# 邻接表（无向图，每条边存两次）
graph = [[1, 2], [0, 2], [0, 1]]    # 0-1, 1-2, 0-2, 构成三角形
print(has_cycle_dfs(graph, 3))      # True
```

三种方法对比

| 方法         | 时间复杂度      | 优势                  |
|------------|------------|---------------------|
| BFS+步数     | O(V+E)     | 不递归，避免栈溢出           |
| 并查集        | ~O(α(n)·E) | 最快，边集表示天然适配         |
| DFS+parent | O(V+E)     | 最直观，可扩展到有向图（改用三色标记） |

二分查找算法

月度开销

```
n,m=map(int,input().split())
expend=[int(input()) for i in range(n)]
def check(x):
    # 判断x作为最大月度开销是否可以实现，如果可以实现，则说明不够小或刚好符合题意。
    # 看m个方案是否可行。
    nums,s=1,0
    for i in range(n):
        if expend[i]+s>x:
            s=expend[i]
            nums+=1    # 求和大于设定的最大月度开销，则应该插入挡板，份数+1
        else:s+=expend[i]
    return nums>m    # if nums>m return True,else return False

lo,hi,res=max(expend),sum(expend)+1,1
while lo<hi:
    mid=(lo+hi)//2
    if check(mid):lo=mid+1
    else:res,hi=mid,mid

print(res)
```

位运算技巧与应用

Python基础语法

| 运算                | 写法                                        | 说明         |
|-------------------|-------------------------------------------|------------|
| 与/或/异或/非          | & \  ^ ~                                  | ~x = -x-1  |
| 左移/右移             | x << k x >> k                             | 乘/除2的k次方   |
| 取第k位（0为最低位）       | (x >> k) & 1                              |            |
| 第k位置1             | x \  (1 << k)                             |            |
| 第k位清0             | x & ~(1 << k)                             |            |
| 第k位翻转             | x ^ (1 << k)                              |            |
| 1的个数              | bin(x).count('1') 或 x.bit_count() (3.10+) |            |
| 二进制位数（log2向下取整+1） | x.bit_length()                            | x=0时返回0    |
| 转二进制串             | bin(x)[2:] 或 f"{x:b}"                     | 切片去掉 0b 前缀 |
| 补0对齐              | f"{x:08b}"                                | 固定8位宽度     |

判奇偶/2的幂/最低位1

```
x & 1          # 奇数为1偶数为0
x > 0 and x & (x-1) == 0  # x是2的幂
x & -x         # lowbit: 取最低位的1对应的值
                # 如 12=0b1100, lowbit(12)=4=0b0100
x & (x - 1)    # 去掉最低位的1（用于Brian Kernighan计1）
```

应用1：树状数组的lowbit

BIT中 tree[i] 覆盖区间 [i-lowbit(i)+1, i]，所以单点更新沿 i += lowbit(i) 往上跳，前缀查询沿 i -= lowbit(i) 往左跳。两者复杂度都是O(logn)。

应用2：状态压缩DP（TSP、斯坦纳树）

用整数mask的二进制位表示"集合"。常见操作：

```
S | (1 << i)    # 加入元素i
S & ~(1 << i)   # 移除元素i
(S >> i) & 1    # 检查元素i是否在集合中
bin(S).count('1') # 集合大小
S == 0         # 空集
S == (1 << n) - 1 # 全集（n个元素）
```

应用3：枚举子集（O(3^n)总复杂度）

```
# 枚举mask的所有非空真子集
sub = (mask - 1) & mask
while sub > 0:
    # 处理 sub 和 mask^sub
    sub = (sub - 1) & mask

# 枚举包括0在内的所有子集
sub = mask
while True:
    # 处理 sub
    if sub == 0: break
    sub = (sub - 1) & mask

# 枚举全集2^n的所有非空子集对(S, mask\S)总复杂度O(3^n)
for mask in range(1 << n):
    sub = mask
    while sub > 0:
        # (sub, mask ^ sub) 互补
        sub = (sub - 1) & mask
```

为什么是O(3^n)而不是O(4^n)：每位有3种状态（不在mask、在mask不在sub、在mask且在sub），总状态数3^n。

应用4：Brian Kernighan算法（数1的个数）

```
def popcount(x):
    c = 0
    while x:
        x &= x - 1      # 每次去掉最低位的1
        c += 1
    return c
# 等价于 bin(x).count('1') 或 x.bit_count()
```

应用5：bit\_length 求log2

```
n.bit_length() - 1 # [log2(n)], 比 int(log2(n)) 更可靠（避免浮点误差）
# 例: 8.bit_length()=4 → log2(8)=3
```

常用于线段树/ST表确定层数：LOG = n.bit\_length()。

## 应用6：状态枚举与子集和DP（SOS DP）

```
# f[S] = sum of a[T] for T ⊆ S
for i in range(n):
    for S in range(1 << n):
        if S & (1 << i):
            f[S] += f[S ^ (1 << i)]    # 加上去掉第i位的贡献
# O(n·2^n)，比朴素的O(3^n)更快
```

## 线段树（Segment Tree）

线段树支持单点修改、区间修改、区间查询（最大/最小/求和），单次操作O(logN)。

通用版（区间和+lazy标记，支持区间加+区间查询）

```
class SegTree:
    def __init__(self, data):
        self.n = len(data)
        self.tree = [0] * (4 * self.n)
        self.lazy = [0] * (4 * self.n)
        self.build(1, 0, self.n - 1, data)

    def build(self, node, l, r, data):
        if l == r:
            self.tree[node] = data[l]
            return
        mid = (l + r) // 2
        self.build(node * 2, l, mid, data)
        self.build(node * 2 + 1, mid + 1, r, data)
        self.tree[node] = self.tree[node * 2] + self.tree[node * 2 + 1]

    def push_down(self, node, l, r):
        if self.lazy[node]:
            mid = (l + r) // 2
            left, right = node * 2, node * 2 + 1
            # 左子树
            self.tree[left] += self.lazy[node] * (mid - l + 1)
            self.lazy[left] += self.lazy[node]
            # 右子树
            self.tree[right] += self.lazy[node] * (r - mid)
            self.lazy[right] += self.lazy[node]
            self.lazy[node] = 0

    # 区间[ql,qr]每个元素加val
    def update(self, node, l, r, ql, qr, val):
        if qr < l or r < ql:
            return
        if ql <= l and r <= qr:
            self.tree[node] += val * (r - l + 1)
            self.lazy[node] += val
            return
        self.push_down(node, l, r)
        mid = (l + r) // 2
        self.update(node * 2, l, mid, ql, qr, val)
        self.update(node * 2 + 1, mid + 1, r, ql, qr, val)
        self.tree[node] = self.tree[node * 2] + self.tree[node * 2 + 1]

    # 查询区间[ql,qr]的和
    def query(self, node, l, r, ql, qr):
        if qr < l or r < ql:
            return 0
        if ql <= l and r <= qr:
            return self.tree[node]
        self.push_down(node, l, r)
        mid = (l + r) // 2
        return (self.query(node * 2, l, mid, ql, qr)
                + self.query(node * 2 + 1, mid + 1, r, ql, qr))

# 使用
data = [1, 3, 5, 7, 9, 11]
st = SegTree(data)
print(st.query(1, 0, st.n - 1, 1, 3)) # 区间[1,3]和=15
st.update(1, 0, st.n - 1, 1, 3, 10) # 区间[1,3]每个加10
print(st.query(1, 0, st.n - 1, 1, 3)) # =45
```

区间最大/最小值线段树（单点更新版）

```
class SegTreeMax:
    def __init__(self, data):
```



```

self.n = len(data)
self.tree = [-float('inf')] * (4 * self.n)
self.build(1, 0, self.n - 1, data)

def build(self, node, l, r, data):
    if l == r:
        self.tree[node] = data[l]
        return
    mid = (l + r) // 2
    self.build(node * 2, l, mid, data)
    self.build(node * 2 + 1, mid + 1, r, data)
    self.tree[node] = max(self.tree[node * 2], self.tree[node * 2 + 1])

# 单点更新：把idx位置的值改为val
def update(self, node, l, r, idx, val):
    if l == r:
        self.tree[node] = val
        return
    mid = (l + r) // 2
    if idx <= mid:
        self.update(node * 2, l, mid, idx, val)
    else:
        self.update(node * 2 + 1, mid + 1, r, idx, val)
    self.tree[node] = max(self.tree[node * 2], self.tree[node * 2 + 1])

# 查询区间[ql,qr]最大值
def query(self, node, l, r, ql, qr):
    if qr < l or r < ql:
        return -float('inf')
    if ql <= l and r <= qr:
        return self.tree[node]
    mid = (l + r) // 2
    return max(self.query(node * 2, l, mid, ql, qr),
               self.query(node * 2 + 1, mid + 1, r, ql, qr))

# 求最小值只要把max换成min，初值改为float('inf')

```

## 树状数组（Binary Indexed Tree / Fenwick Tree）

只支持单点修改与前缀查询，比线段树短得多。lowbit(x) = x & -x。下标必须从1开始（lowbit(0)=0会死循环）。

**基础模板（无class，全局数组+函数）**

```

n = 10                # 数组长度
tree = [0] * (n + 1)  # 下标1开始

def lowbit(x):
    return x & -x

def update(idx, val):    # 位置idx增加val
    while idx <= n:
        tree[idx] += val
        idx += lowbit(idx)

def query(idx):          # 前缀[1,idx]之和
    s = 0
    while idx > 0:
        s += tree[idx]
        idx -= lowbit(idx)
    return s

def range_query(l, r):   # 区间[l,r]之和
    return query(r) - query(l - 1)

```

**应用1：求逆序对数（完整程序）**

```

样例输入
8
5 3 7 1 9 6 4 2
样例输出
17

```

思想：从右往左扫，对当前数a[i]，查询树状数组中已经出现过且严格小于a[i]的个数（即a[i]右侧比它小的元素个数），累加即为逆序对数。值域大时先离散化把值映射到1~m。

```

n = int(input())
a = list(map(int, input().split()))

# 离散化：把a中元素映射到1..m
sorted_a = sorted(set(a))

```

```

rank = {v: i + 1 for i, v in enumerate(sorted_a)}
m = len(sorted_a)

tree = [0] * (m + 1)

def lowbit(x):
    return x & -x

def update(idx, val):
    while idx <= m:
        tree[idx] += val
        idx += lowbit(idx)

def query(idx):
    s = 0
    while idx > 0:
        s += tree[idx]
        idx -= lowbit(idx)
    return s

inv = 0
# 从右往左扫，每次查询比a[i]严格小的、已出现的元素个数
for x in reversed(a):
    r = rank[x]
    inv += query(r - 1)
    update(r, 1)

print(inv)

```

## 应用2：单点修改+区间求和（完整程序）

最经典的BIT用法。支持两种操作：(1) 把a[i]加上val；(2) 查询区间[l,r]之和。

样例输入

```

5
1 2 3 4 5
5
2 1 3
1 3 5
2 1 3
2 4 5
2 1 5

```

样例输出

```

6
11
9
20

```

```

n = int(input())
a = list(map(int, input().split()))

tree = [0] * (n + 1)

def lowbit(x):
    return x & -x

def update(idx, val):
    while idx <= n:
        tree[idx] += val
        idx += lowbit(idx)

def query(idx):
    s = 0
    while idx > 0:
        s += tree[idx]
        idx -= lowbit(idx)
    return s

# 初始化：每个位置当作单点更新加进去
for i in range(n):
    update(i + 1, a[i])

q = int(input())
for _ in range(q):
    op = list(map(int, input().split()))
    if op[0] == 1:
        # 1 i val: a[i]+=val
        i, val = op[1], op[2]
        update(i, val)
    else:
        # 2 l r: 查询[l,r]之和
        l, r = op[1], op[2]
        print(query(r) - query(l - 1))

```

### 应用3：区间修改+单点查询（差分数组+BIT，完整程序）

思想：维护差分序列 $\text{diff}[i]=a[i]-a[i-1]$ ，则 $a[i]=\text{diff}$ 前缀和。区间 $[l,r]$ 加 $\text{val}$ 只需 $\text{diff}[l]+=val$ ,  $\text{diff}[r+1]-=val$ （两次单点更新）。BIT存的是 $\text{diff}$ 。

```
样例输入
5
1 2 3 4 5
4
1 2 4 10
2 3
1 1 5 1
2 1
样例输出
13
2
```

```
n = int(input())
a = list(map(int, input().split()))

tree = [0] * (n + 2)          # +2 防止r+1越界

def lowbit(x):
    return x & -x

def update(idx, val):
    while idx <= n:
        tree[idx] += val
        idx += lowbit(idx)

def query(idx):
    s = 0
    while idx > 0:
        s += tree[idx]
        idx -= lowbit(idx)
    return s

# 初始化：把原数组的差分序列加入BIT
prev = 0
for i in range(n):
    update(i + 1, a[i] - prev)
    prev = a[i]

q = int(input())
for _ in range(q):
    op = list(map(int, input().split()))
    if op[0] == 1:              # 1 l r val: 区间[l,r]每个加val
        l, r, val = op[1], op[2], op[3]
        update(l, val)
        update(r + 1, -val)
    else:                       # 2 i: 查询a[i]
        i = op[1]
        print(query(i))
```

### LCA（最小公共祖先，倍增法）

预处理 $O(N\log N)$ ，每次查询 $O(\log N)$ 。up[k][v] 表示v的第 $2^k$ 个祖先。

```
import sys
from math import log2
sys.setrecursionlimit(200000)

class LCA:
    def __init__(self, n, root, graph):
        self.n = n
        self.LOG = max(1, int(log2(n)) + 1)
        self.depth = [0] * n
        self.up = [[-1] * n for _ in range(self.LOG)]
        self.graph = graph
        self.dfs(root, -1, 0)
        # 倍增预处理
        for k in range(1, self.LOG):
            for v in range(n):
                if self.up[k - 1][v] != -1:
                    self.up[k][v] = self.up[k - 1][self.up[k - 1][v]]

    def dfs(self, u, parent, d):
        self.up[0][u] = parent
        self.depth[u] = d
        for v in self.graph[u]:
            if v != parent:
                self.dfs(v, u, d + 1)
```

```
def query(self, u, v):
    # 让u是更深的那个
    if self.depth[u] < self.depth[v]:
        u, v = v, u
    diff = self.depth[u] - self.depth[v]
    # u往上跳diff步
    for k in range(self.LOG):
        if (diff >> k) & 1:
            u = self.up[k][u]
    if u == v:
        return u
    # 同时往上跳直到父亲相同
    for k in range(self.LOG - 1, -1, -1):
        if self.up[k][u] != self.up[k][v]:
            u = self.up[k][u]
            v = self.up[k][v]
    return self.up[0][u]

# 邻接表graph[i]=[邻居们]，建好后：
# lca = LCA(n, root, graph)
# print(lca.query(u, v))
```

## 二叉树基本概念

### 几种特殊二叉树

| 类型                 | 定义                                                  | 关键性质                             |
|--------------------|-----------------------------------------------------|----------------------------------|
| 满二叉树（Full BT）      | 每个节点要么是叶子要么有2个孩子                                    | 叶节点数 = 内部节点数+1                   |
| 完美二叉树（Perfect BT）  | 所有内部节点都有2个孩子，且所有叶子在同一层                              | $n = 2^h - 1$ （h为高度+1）           |
| 完全二叉树（Complete BT） | 除最后一层外其余层全满，最后一层从左到右连续填充                            | 可用数组紧凑存储；堆就是完全BT                 |
| 二叉搜索树（BST）         | 左子树所有值 < 根 < 右子树所有值                                 | 中序遍历是升序序列                        |
| 平衡二叉树（AVL）         | 每个节点 $ \text{height}(L) - \text{height}(R)  \leq 1$ | 高度 $O(\log n)$ ，所有操作 $O(\log n)$ |

注：中文教材里"满二叉树"有时指 Perfect BT（每层都满），术语易混淆。题目中以题面定义为准。

### 重要性质（节点数为n、高度为h，根节点高度=0）

- 第 i 层最多  $2^i$  个节点（i从0数）
- 高度为 h 的二叉树最多  $2^{(h+1)} - 1$  个节点
- n 个节点的二叉树高度至少为  $\lceil \log_2(n+1) \rceil - 1$ ，至多为  $n-1$
- 任意二叉树中：叶子数  $n_0$  = 度为2的节点数  $n_2 + 1$ （常考结论）
- n 个节点的完全二叉树高度 =  $\lceil \log_2(n) \rceil$

### 完全二叉树的数组表示（堆的实现基础）

下标从 0 开始时：节点 i 的左孩  $2i+1$ ，右孩  $2i+2$ ，父亲  $(i-1)//2$ 。  
下标从 1 开始时：节点 i 的左孩  $2i$ ，右孩  $2i+1$ ，父亲  $i//2$ （heap常用）。

```
def is_complete(root):
    """BFS判定完全二叉树：见过None后不应再有节点"""
    if not root: return True
    dq = deque([root])
    seen_none = False
    while dq:
        node = dq.popleft()
        if node is None:
            seen_none = True
        else:
            if seen_none: return False # None后还有节点 → 不完全
            dq.append(node.left)
            dq.append(node.right)
    return True
```

### BST 基本操作

```
def bst_insert(root, val):
    if not root: return TreeNode(val)
    if val < root.val: root.left = bst_insert(root.left, val)
    elif val > root.val: root.right = bst_insert(root.right, val)
    return root

def bst_search(root, val):
    while root and root.val != val:
        root = root.left if val < root.val else root.right
    return root
```

```
def is_bst(root, lo=float('-inf'), hi=float('inf')):
    if not root: return True
    if not (lo < root.val < hi): return False
    return is_bst(root.left, lo, root.val) and is_bst(root.right, root.val, hi)
```

**AVL 旋转思想（了解即可）**：插入后若某节点失衡（左右高差>1），按失衡情形做 LL/RR/LR/RL 四种旋转。竞赛中通常用 `heapq` 或 `SortedList` 替代手写 AVL。

**树的高度/直径**

```
def height(root):
    if not root: return -1          # 空树高度-1, 单节点高度0
    return 1 + max(height(root.left), height(root.right))

def diameter(root):
    """直径=任意两节点最长路径的边数"""
    ans = 0
    def depth(node):
        nonlocal ans
        if not node: return 0
        l = depth(node.left)
        r = depth(node.right)
        ans = max(ans, l + r)        # 经过当前节点的最长路径
        return 1 + max(l, r)
    depth(root)
    return ans
```

**树的遍历（前序、中序、后序、层序）**

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

# 递归版
def preorder(root):    # 根 左 右
    if not root: return []
    return [root.val] + preorder(root.left) + preorder(root.right)

def inorder(root):     # 左 根 右
    if not root: return []
    return inorder(root.left) + [root.val] + inorder(root.right)

def postorder(root):   # 左 右 根
    if not root: return []
    return postorder(root.left) + postorder(root.right) + [root.val]

# 迭代版（用栈）
def preorder_iter(root):
    if not root: return []
    stack, res = [root], []
    while stack:
        node = stack.pop()
        res.append(node.val)
        if node.right: stack.append(node.right) # 先压右
        if node.left: stack.append(node.left)
    return res

def inorder_iter(root):
    stack, res, cur = [], [], root
    while cur or stack:
        while cur:
            stack.append(cur)
            cur = cur.left
        cur = stack.pop()
        res.append(cur.val)
        cur = cur.right
    return res

def postorder_iter(root):
    if not root: return []
    stack, res = [root], []
    while stack:
        node = stack.pop()
        res.append(node.val)                # 根右左 反过来就是 左右根
        if node.left: stack.append(node.left)
        if node.right: stack.append(node.right)
    return res[::-1]

# 层序遍历（BFS）
from collections import deque
```

```
def levelorder(root):
    if not root: return []
    dq, res = deque([root]), []
    while dq:
        level = []
        for _ in range(len(dq)):
            node = dq.popleft()
            level.append(node.val)
            if node.left: dq.append(node.left)
            if node.right: dq.append(node.right)
        res.append(level) # 想要扁平直接 res.extend(level)
    return res
```

### 已知前序+中序建树

```
def build(preorder, inorder):
    if not preorder: return None
    root = TreeNode(preorder[0])
    idx = inorder.index(preorder[0])
    root.left = build(preorder[1:1+idx], inorder[:idx])
    root.right = build(preorder[1+idx:], inorder[idx+1:])
    return root
```

### 已知后序+中序建树

```
def build_post(postorder, inorder):
    if not postorder: return None
    root = TreeNode(postorder[-1])
    idx = inorder.index(postorder[-1])
    root.left = build_post(postorder[:idx], inorder[:idx])
    root.right = build_post(postorder[idx:-1], inorder[idx+1:])
    return root
```

### 已知层序+中序建树

层序中第一个出现的节点是当前子树的根。在中序中找到它，将中序左右分割；层序中保留属于左/右子树的节点（用集合过滤）即可递归。

```
def build_level(levelorder, inorder):
    if not inorder: return None
    # 层序中第一个在inorder中的就是根
    root_val = next(v for v in levelorder if v in inorder)
    root = TreeNode(root_val)
    idx = inorder.index(root_val)
    left_in, right_in = inorder[:idx], inorder[idx+1:]
    left_set, right_set = set(left_in), set(right_in)
    left_level = [v for v in levelorder if v in left_set]
    right_level = [v for v in levelorder if v in right_set]
    root.left = build_level(left_level, left_in)
    root.right = build_level(right_level, right_in)
    return root
```

### 遍历之间的直接转换（不显式建树）

只要能根+左右分割，就能在递归过程中直接拼出目标遍历，省下建树步骤。

```
# 前序+中序 → 后序
def pre_in_to_post(pre, ino):
    if not pre: return []
    root = pre[0]
    idx = ino.index(root)
    left = pre_in_to_post(pre[1:1+idx], ino[:idx])
    right = pre_in_to_post(pre[1+idx:], ino[idx+1:])
    return left + right + [root] # 左 右 根

# 后序+中序 → 前序
def post_in_to_pre(post, ino):
    if not post: return []
    root = post[-1]
    idx = ino.index(root)
    left = post_in_to_pre(post[:idx], ino[:idx])
    right = post_in_to_pre(post[idx:-1], ino[idx+1:])
    return [root] + left + right # 根 左 右

# 前序+中序 → 层序（先建树后BFS）
def pre_in_to_level(pre, ino):
    root = build(pre, ino) # 复用上面的build
    return levelorder(root)
```

**唯一性：**前序+中序、后序+中序、层序+中序都能**唯一**确定二叉树。前序+后序**不能**唯一确定（除非每个内部节点都有2个孩子，即满二叉树）。

## 遍历推导口诀

- 前序首元素 / 后序末元素 = 当前子树的根
- 中序中根的位置 = 左右子树的分界
- 层序中第一个 = 整树根；除去当前根后**保持原相对顺序**的部分是左/右子树的层序

**BST 由前序重建**（无需中序，因为中序=排序后的前序）

```
def bst_from_pre(pre):
    if not pre: return None
    root = TreeNode(pre[0])
    # 找到第一个大于根的位置 → 右子树起点
    i = 1
    while i < len(pre) and pre[i] < pre[0]:
        i += 1
    root.left = bst_from_pre(pre[1:i])
    root.right = bst_from_pre(pre[i:])
    return root
```

## Trie（字典树/前缀树）

用嵌套dict实现最简洁：每个节点是一个dict，键是字符，值是子节点；用特殊键（如 '#'）标记单词结尾。

**基础操作**：insert/search/starts\_with 都是O(L)，L是单词长度。

```
class Trie:
    def __init__(self):
        self.root = {}

    def insert(self, word):
        node = self.root
        for c in word:
            if c not in node:
                node[c] = {}
            node = node[c]
        node['#'] = True  # 单词结尾标记

    def search(self, word):  # 精确查找
        node = self.root
        for c in word:
            if c not in node:
                return False
            node = node[c]
        return '#' in node

    def starts_with(self, prefix):  # 前缀查询
        node = self.root
        for c in prefix:
            if c not in node:
                return False
            node = node[c]
        return True
```

**典型应用：电话号码一致性（OJ04089）**

给若干电话号码，判断是否存在一个号码是另一个的前缀（如91、9112 → 91是9112的前缀，输出NO）。

思路：把号码按长度排序，依次insert；插入第i个号码时若走到某节点已经有 '#' 标记，则它是某个已有号码的延伸→冲突；或走完它后子树非空也是冲突（它是前面某号码的前缀）。

```
def is_consistent(numbers):
    numbers.sort(key=len)  # 短的先插入
    root = {}
    for num in numbers:
        node = root
        for c in num:
            if '#' in node:  # 路径上已有完整号码 → 它是当前号码前缀
                return False
            if c not in node:
                node[c] = {}
            node = node[c]
        if node:  # 当前号码插完，节点还有子树 → 它是别人的前缀
            return False
        node['#'] = True
    return True

# 示例
print(is_consistent(["911", "97625999", "91125426"]))  # False (911是91125426前缀)
print(is_consistent(["113", "12340", "123440", "12345", "98346"]))  # True
```

其他典型应用：

- **自动补全**：在节点上额外存所有以此为前缀的单词或排名最高的几个
- **字符串集合最长公共前缀**：从根走到第一个分支点或带 '#' 的节点
- **统计前缀出现次数**：每个节点加 count 字段，insert时一路+1
- **01-Trie求异或最大值**：把整数按二进制位插入Trie，查询时贪心走相反位

## 大顶堆（Max Heap）

Python的heapq是小顶堆。最简单的大顶堆做法是**取负数**：入堆 `heappush(heap, -x)`，出堆 `-heappop(heap)`。

```
import heapq
heap = []
for x in [3, 1, 4, 1, 5, 9, 2, 6]:
    heapq.heappush(heap, -x)
print(-heapq.heappop(heap))    # 9
```

对元组等无法取负的场景，可定义类并重载 `__lt__` 反向：

```
class Item:
    def __init__(self, val): self.val = val
    def __lt__(self, other): return self.val > other.val    # 反向
```

## KMP的LPS数组及应用

LPS（Longest Proper Prefix which is also Suffix）：lps[i]表示模式串s[0..i]的最长"既是真前缀又是真后缀"的长度。

**构造LPS（O(N)）**

```
def build_lps(s):
    n = len(s)
    lps = [0] * n
    length = 0    # 当前匹配的前缀长度
    i = 1
    while i < n:
        if s[i] == s[length]:
            length += 1
            lps[i] = length
            i += 1
        else:
            if length != 0:
                length = lps[length - 1]    # 关键回退
            else:
                lps[i] = 0
                i += 1
    return lps

# 例: s='ababcababa'
# lps  = [0,0,1,2,0,1,2,3,4,3]
```

**应用1：在文本text中查找模式pattern的所有出现位置（KMP匹配）**

```
def kmp_search(text, pattern):
    if not pattern: return []
    lps = build_lps(pattern)
    res, i, j = [], 0, 0    # i:text指针, j:pattern指针
    while i < len(text):
        if text[i] == pattern[j]:
            i += 1; j += 1
            if j == len(pattern):
                res.append(i - j)
                j = lps[j - 1]    # 继续找下一处
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res

print(kmp_search("ababcbababcbab", "ababc"))    # [0, 6]
```

**应用2：求字符串的最小循环节（最小周期）**

定理：设字符串长n，lps[n-1]=k。若  $n = (md(n))$ ，则最小周期为n-k，最小循环节就是s[:n-k]，整个串由它重复n/(n-k)次组成；否则字符串不能由完整循环节重复构成，"广义最小周期"仍为n-k。

```
def min_period(s):
    n = len(s)
    lps = build_lps(s)
```



```

p = n - lps[n - 1]
if n % p == 0:
    return p, n // p    # 周期长度, 循环次数
return p, None          # 不能完整循环, 仅是广义周期

print(min_period("abcabcabc"))    # (3, 3)
print(min_period("abcabca"))      # (3, None) - abc重复但末尾不全

```

应用3: 判断t是否为s的子串只需调用kmp\_search

## 旅行商问题 (TSP, Traveling Salesman Problem)

**问题描述:** 给定n个城市和两两之间的距离矩阵  $dist[i][j]$ , 求从起点出发、访问每个城市恰好一次、最后回到起点的最短路径长度。TSP是NP-hard问题, 没有多项式算法。

**状压DP (Held-Karp,  $n \leq 20$ ):**  $dp[S][i]$  表示已访问集合为S (二进制位)、停在i的最短路。

状态转移:  $dp[i] = \min_j (dp[j] + dist[j][i])$

答案:  $dp[全集合][i] + dist[i][0]$ 。时间 $O(n^2 \cdot 2^n)$ , 空间 $O(n \cdot 2^n)$ 。

```

def tsp_dp(dist):
    n = len(dist)
    INF = float('inf')
    dp = [[INF] * n for _ in range(1 << n)]
    dp[1][0] = 0                                # 起点0已访问

    for S in range(1 << n):
        if not (S & 1): continue
        for i in range(n):
            if not (S >> i) & 1: continue
            if dp[S][i] == INF: continue
            for j in range(n):
                if (S >> j) & 1: continue      # j不能已访问
                new_S = S | (1 << j)
                cost = dp[S][i] + dist[i][j]
                if cost < dp[new_S][j]:
                    dp[new_S][j] = cost

    full = (1 << n) - 1
    return min(dp[full][i] + dist[i][0] for i in range(1, n))

# 示例
dist = [[0,10,15,20],[10,0,35,25],[15,35,0,30],[20,25,30,0]]
print(tsp_dp(dist))    # 80 (路径 0→1→3→2→0)

```

**还原路径:** 开  $parent[S][i]$  记录到达 $dp[S][i]$ 时的上一个城市, 从  $full, best\_end$  倒推。

**完整优化版 (OJ提交模板, 含IO+剪枝+常数优化)**

针对Python运行慢的特性, 做四类优化:

- 快速I/O:** `sys.stdin.read().split()` 一次读完 + `iter/next`, 比循环 `input()` 快10倍以上
- 数据结构选择:** 用二维list代替dict存图 (list按内存偏移直接访问, dict要哈希)
- 逻辑剪枝:**
  - `range(1, size, 2)`: 从0出发→mask第0位必为1→mask必为奇数, 步长2省一半循环
  - 内层 `range(1, n)`: 中间路径不能回到起点0
- 常数优化 (Python特有):**
  - 去掉 `min()` 函数调用:** 函数调用栈开销大, 千万级循环里改用 `if val < dp[...]: dp[...] = val` 加速明显
  - 局部变量缓存:** `curr = dp[mask][u]` 避免多次二维索引

```

import sys

def solve():
    data = sys.stdin.read().split()
    it = iter(data)
    try:
        n = int(next(it))
    except StopIteration:
        return

    # 二维list存距离矩阵
    dist = [[int(next(it)) for _ in range(n)] for _ in range(n)]

    size = 1 << n
    INF = float('inf')
    dp = [[INF] * n for _ in range(size)]
    dp[1][0] = 0                                # 起点固定0, 初始mask=1

    # 关键优化: 只遍历奇数mask (第0位必为1)
    for i in range(1, size, 2):
        for j in range(n):

```

```

curr = dp[i][j] # 局部变量缓存
if curr == INF:
    continue
# 内层k从1开始（中间过程不回0）
for k in range(1, n):
    if (i >> k) & 1: # k已在集合中
        continue
    new_mask = i | (1 << k)
    new_cost = curr + dist[j][k]
    # 手动比较替代min()
    if new_cost < dp[new_mask][k]:
        dp[new_mask][k] = new_cost

# 回到起点0
ans = INF
full = size - 1
for i in range(1, n):
    c = dp[full][i] + dist[i][0]
    if c < ans:
        ans = c
print(ans)

if __name__ == '__main__':
    solve()

```

优化效果说明：n=15时，朴素Python约4-6秒，上面所有优化叠加后通常能压到1秒内通过。

**DFS+剪枝（n稍大时备选）**：用当前最优解作上界剪枝，最坏 $O(n!)$ 但常数小。

**最近邻贪心（n很大时）**： $O(n^2)$ ，结果通常比最优多20-25%。其他启发式：2-opt、模拟退火、遗传算法等。

**变形题型：**

- **路径TSP**（不回起点）：答案改为  $\min(dp[full][i])$
- **带起终点约束**：固定起点终点
- **TSP计数**：min改sum，求方案数

## 经典题目：Top2招生工作（OJ28749）

**题意简述**：n个学生初始颜色R/P/W，m个志愿者各负责一组学生。每次某志愿者"沟通"会让其负责的所有学生颜色循环+1（ $R \rightarrow P \rightarrow W \rightarrow R$ ）。求让所有学生变R所需的最少沟通次数，无解输出 impossible。

**建模**：颜色映射  $R=0, P=1, W=2$ 。把志愿者视作节点（编号1..m），每个学生看作连接其两个志愿者的"边"；只连一个志愿者的学生用**虚拟节点0**作另一端。设  $x_v$  为志愿者v的沟通次数（取值0/1/2即可，因为3次=不沟通）。每个学生的方程：

$$x + x \pmod{3} = c$$

**求解**：在模3下逐连通分量处理。

- 含虚拟节点0的分量：固定  $x_0=0$ ，BFS唯一确定所有节点取值（重边必须一致，否则 impossible）
- 不含0的分量：枚举根节点  $x_{root} \in \{0, 1, 2\}$  三种取值，BFS推出整个分量，取使分量内总和最小的方案
- 全部加和即答案

**特殊情形**：孤立学生（没有任何志愿者负责）必须初始为R，否则 impossible。

```

import sys
from collections import deque

def solve() -> None:
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    it = iter(data)
    n = int(next(it))
    m = int(next(it))
    color_str = next(it).decode()

    # 0:R, 1:P, 2:W
    init = [0] * (n + 1)
    for i, ch in enumerate(color_str, 1):
        if ch == 'R':
            init[i] = 0
        elif ch == 'P':
            init[i] = 1
        else:
            # 'W'
            init[i] = 2

    # 每个学生关联的志愿者列表（至多两个）
    stu_vol = [[] for _ in range(n + 1)]
    for vol in range(1, m + 1):
        k = int(next(it))
        for _ in range(k):
            stu = int(next(it))

```

```

        stu_vol[stu].append(vol)

# 无志愿者联系的学生必须初始为红色
for stu in range(1, n + 1):
    if not stu_vol[stu] and init[stu] != 0:
        print("impossible")
        return

# 建图: 志愿者 0..m
graph = [[] for _ in range(m + 1)]          # graph[u] = [(v, target), ...]
for stu in range(1, n + 1):
    lst = stu_vol[stu]
    if not lst:
        continue
    if len(lst) == 1:
        u, v = lst[0], 0
    else:
        u, v = lst[0], lst[1]
    target = (3 - init[stu]) % 3
    # 无向边, 双向添加
    graph[u].append((v, target))
    graph[v].append((u, target))

visited = [False] * (m + 1)
total_cost = 0

# ----- 处理包含虚拟节点 0 的分量 -----
if graph[0]:
    comp_nodes = []
    q = deque([0])
    visited[0] = True
    while q:
        u = q.popleft()
        comp_nodes.append(u)
        for v, _ in graph[u]:
            if not visited[v]:
                visited[v] = True
                q.append(v)

    assign = {0: 0}
    q = deque([0])
    ok = True
    while q and ok:
        u = q.popleft()
        val_u = assign[u]
        for v, target in graph[u]:
            val_v = (target - val_u) % 3
            if v not in assign:
                assign[v] = val_v
                q.append(v)
            elif assign[v] != val_v:
                ok = False
                break
    if not ok:
        print("impossible")
        return

    for node in comp_nodes:
        if node != 0:
            total_cost += assign[node]

# ----- 处理剩余不含 0 的分量 -----
for start in range(1, m + 1):
    if visited[start]:
        continue

    # BFS 收集分量内所有节点 (不含0)
    comp_nodes = []
    q = deque([start])
    visited[start] = True
    comp_nodes.append(start)
    while q:
        u = q.popleft()
        for v, _ in graph[u]:
            if v != 0 and not visited[v]:
                visited[v] = True
                q.append(v)
                comp_nodes.append(v)

    best = None
    for root_val in range(3):
        assign = {start: root_val}
        q = deque([start])

```

```

    ok = True
    while q and ok:
        u = q.popleft()
        val_u = assign[u]
        for v, target in graph[u]:
            if v == 0:
                continue
            val_v = (target - val_u) % 3
            if v not in assign:
                assign[v] = val_v
                q.append(v)
            elif assign[v] != val_v:
                ok = False
                break

    if ok:
        cost = sum(assign[node] for node in comp_nodes)
        if best is None or cost < best:
            best = cost

    if best is None:
        print("impossible")
        return
    total_cost += best

    print(total_cost)

if __name__ == "__main__":
    solve()

```

关键技巧总结：

- **状态空间限制**：模3下每个变量只有3种取值，可枚举
- **约束传播**：连通分量内固定一个变量后其他全部确定
- **虚拟节点**：把“只有一端”的边补成完整边便于统一处理
- **重边一致性检查**：BFS推导时已访问节点的值必须与新推导值一致，否则无解

## 数值格式化与精度

保留小数位（最常用）

```

x = 3.14159265

# f-string (推荐, Python 3.6+)
print(f"{x:.2f}")          # 3.14    定点2位小数
print(f"{x:.4f}")          # 3.1416
print(f"{x:10.2f}")        # '      3.14'   宽度10右对齐
print(f"{x:<10.2f}")       # '3.14      '   左对齐
print(f"{x:010.2f}")       # 0000003.14   补0对齐
print(f"{x:+.2f}")         # +3.14    强制带正负号
print(f"{x:.2e}")          # 3.14e+00   科学计数法
print(f"{x:.2%}")          # 314.16%    百分号格式

# format方法 (旧代码常见)
print("{:.2f}".format(x))   # 3.14
print("{:.2f} 和 {:.3f}".format(x, x)) # 多参数

# %格式化 (C风格, 最旧)
print("%.2f" % x)          # 3.14
print("%6.2f" % x)         # '      3.14'   宽度6

# round函数: 返回数值 (不是字符串)
print(round(x, 2))         # 3.14
print(round(2.5))          # 2    ⚠️ 银行家舍入: 偶数优先
print(round(3.5))          # 4

```

整数格式化

```

n = 42
print(f"{n:5d}")          # '      42'   宽度5
print(f"{n:05d}")         # '00042'   补0
print(f"{n:b}")           # '101010'   二进制
print(f"{n:o}")           # '52'        八进制
print(f"{n:x}")           # '2a'        十六进制小写
print(f"{n:X}")           # '2A'        十六进制大写
print(f"{n:#b}")          # '0b101010' 带前缀
print(f"{n:,}")           # 千分位 1,234,567

```

浮点数陷阱与判等

```

0.1 + 0.2 == 0.3      # False! 实际是 0.30000000000000004
abs(0.1 + 0.2 - 0.3) < 1e-9 # 正确做法: 用eps判等

# 显示也有陷阱
print(0.1 + 0.2)      # 0.30000000000000004
print(f"{0.1+0.2:.2f}") # 0.30 格式化时已四舍五入

# 高精度浮点: decimal模块
from decimal import Decimal, getcontext
getcontext().prec = 50
Decimal('0.1') + Decimal('0.2') # Decimal('0.3')

```

## 整数除法、取模、幂

```

17 // 5      # 3      整除（向下取整，负数也是）
-17 // 5     # -4     ⚠️ 不是-3! 向下取整对负数会变小
17 % 5       # 2      取模（结果符号与除数相同）
-17 % 5      # 3      ⚠️ 不是-2
divmod(17, 5) # (3, 2) 一次得商和余

# 整数幂（Python整数无溢出）
2**100      # 1267650600228229401496703205376
pow(2, 10, 1000) # 24 带模数的快速幂，比 2**10 % 1000 高效

# 浮点除法
17 / 5      # 3.4

# 向上取整
import math
math.ceil(17 / 5) # 4
-(-17 // 5)      # 4 不用math的整数写法
(17 + 5 - 1) // 5 # 4 等价整数写法

```

## 类型转换

```

int("42")      # 42
int("0b101", 2) # 5 带进制
int("ff", 16)  # 255
int(3.9)       # 3 截断（不是四舍五入！）
int(-3.9)      # -3 向0截断
float("3.14")   # 3.14
float("inf")     # 无穷大
float("nan")     # 非数（任何比较都返回False）
str(3.14)       # '3.14'

```

## 输出技巧

```

# 多个数字按空格输出，每个保留2位小数
nums = [1.0, 2.5, 3.14159]
print(' '.join(f"{x:.2f}" for x in nums)) # 1.00 2.50 3.14

# 矩阵格式化
matrix = [[1.1, 2.22], [3.333, 4.4444]]
for row in matrix:
    print(' '.join(f"{x:8.3f}" for x in row))

```

## bisect 模块（有序序列二分查找）

**bisect** 在已排序列上做O(logN)二分查找/插入。常用于：维护有序集合、求第k小、计算排名、离散化、二分答案的辅助函数。

### 核心API

```

import bisect

a = [1, 3, 3, 5, 7, 9]

# 1. bisect_left(a, x): 返回x应插入的最左位置（已存在x则返回最左x的下标）
bisect.bisect_left(a, 3) # 1 第一个3的位置
bisect.bisect_left(a, 4) # 3 4应插在下标3处

# 2. bisect_right(a, x) 或 bisect(a, x): 返回x应插入的最右位置
bisect.bisect_right(a, 3) # 3 最后一个3之后的位置
bisect.bisect(a, 4)      # 3 bisect 是 bisect_right 的别名

# 3. insort_left / insort_right: 插入并保持有序，O(logN)查找+O(N)移动
bisect.insort(a, 4)      # a变为 [1,3,3,4,5,7,9] 默认右插
bisect.insort_left(a, 3) # 在最左3处插入

```

```
# 指定范围 [lo, hi)
bisect.bisect_left(a, 3, lo=2, hi=5)
```

### lo/hi 参数 + key 参数 (Python 3.10+)

```
# 按对象的某个键二分 (无需手动提取关键字列表)
pairs = [(1, 'a'), (3, 'b'), (5, 'c'), (7, 'd')]
bisect.bisect_left(pairs, 4, key=lambda p: p[0])    # 2
```

### 应用1: 查找元素是否存在

```
def contains(a, x):
    i = bisect.bisect_left(a, x)
    return i < len(a) and a[i] == x
```

### 应用2: 找 $\leq x$ / $< x$ / $\geq x$ / $> x$ 的元素

```
# 已排序 a
# 第一个 >= x 的位置:    bisect_left(a, x)
# 第一个 > x 的位置:     bisect_right(a, x)
# 最后一个 <= x 的位置:  bisect_right(a, x) - 1
# 最后一个 < x 的位置:   bisect_left(a, x) - 1

# 统计区间 [lo, hi) 内元素个数
def count_in_range(a, lo, hi):
    return bisect.bisect_right(a, hi) - bisect.bisect_left(a, lo)

# 统计 == x 的个数
def count_eq(a, x):
    return bisect.bisect_right(a, x) - bisect.bisect_left(a, x)
```

### 应用3: 离散化 (坐标压缩)

```
a = [100, 5, 1000, 5, 50]
sorted_uniq = sorted(set(a))
rank = [bisect.bisect_left(sorted_uniq, x) for x in a]
# rank = [2, 0, 3, 0, 1]    原值映射到 0..k-1
```

### 应用4: 最长上升子序列 LIS ( $O(N \log N)$ )

经典应用: 维护"长度为i的LIS末尾最小值"数组 d, 用 bisect\_left 找替换位置。

```
def length_of_LIS(nums):
    d = []
    for x in nums:
        i = bisect.bisect_left(d, x)    # 严格上升用 bisect_left
        if i == len(d):
            d.append(x)
        else:
            d[i] = x
    return len(d)

print(length_of_LIS([10,9,2,5,3,7,101,18]))    # 4 (2,3,7,101)

# 非严格上升 (允许相等) 用 bisect_right
# 求最长下降序列: 先取负数再求LIS
```

### 应用5: 在线维护有序序列

```
sorted_list = []
for x in stream:
    bisect.insort(sorted_list, x)    # 每次插入 O(N)
    median = sorted_list[len(sorted_list)//2]
```

注: insort 总体  $O(N^2)$ , 大数据需要时用 SortedList (来自第三方库 sortedcontainers, OJ通常不可用) 或线段树/树状数组。

### bisect\_left vs bisect\_right 速记

| 函数                | x存在时返回      | 用途                  |
|-------------------|-------------|---------------------|
| bisect_left(a,x)  | x第一次出现的下标   | "第一个 $\geq x$ " 的位置 |
| bisect_right(a,x) | x最后一次出现的下一位 | "第一个 $> x$ " 的位置    |

两者相减即 count(x)。

## Python考试通用陷阱与优化清单

一、字符串相关

- 字符串比较是按字典序（逐字符ASCII/Unicode）。例 "12" < "2" 为 True，但数值上 12 > 2。需要数值比较时先 int() 转换。
- 字符串与数字拼接需显式转换： "结果" + str(123)。
- strip() 只去首尾空白；split()（无参）按任意空白分割（多空格、\t、\n），split(' ') 只按单个空格分割。

二、递归深度

- Python默认递归深度约1000，大数据（树深10^6）必爆 RecursionError。
  - sys.setrecursionlimit(10\*\*6) —— 仍受C栈限制，>10^6可能段错误
  - 优先改迭代：显式栈模拟DFS、队列做BFS、并查集用循环路径压缩
- 函数不要用可变默认参数：def f(lst=[]) 会跨调用累积。

三、输入输出优化（大数据量）

```
import sys
data = sys.stdin.buffer.read().split()
it = iter(data)
n = int(next(it))
```

- 多次输出时收集到list最后 '\n'.join(...) 一次性print，或用 sys.stdout.write。
- 循环内频繁 print 会拖慢一两个数量级。

四、列表与内存

- [[0]\*n]\*m 错！会得到m行引用同一子列表。正确：[[0]\*n for \_ in range(m)]。
- 删除列表元素正序删会乱索引，要倒序删或列表推导式。
- 频繁删头部：用 collections.deque，list的 pop(0) 是O(n)。

五、字典默认值

- collections.defaultdict(int/list/set) 比反复判key存在要简洁高效。
- 计数：collections.Counter。

六、排序与比较

- sorted(d.items(), key=lambda kv: kv[1]) 按值排字典。
- 自定义复杂排序用 functools.cmp\_to\_key。
- Python的 sort 是稳定的（相等元素保持原顺序）。

七、集合

- in set O(1), in list O(n)。大数据查询多用 set。
- 集合运算：| (并) & (交) - (差) ^ (对称差)。

八、常见坑

- input().split() 得字符串列表，记得 map(int, ...) 转换。
- 邻接表： [[] for \_ in range(n)]，**不要用** [[]] \* n。
- 浮点比较：abs(a - b) < 1e-9，不要用 ==。
- 闭包陷阱：循环中 lambda 捕获循环变量按引用，需 lambda i=i: ... 固定。
- 修改全局可变对象（list/dict）不需 global，修改不可变（int/str）必须 global。

九、常用模块速查

| 模块/函数                                       | 用途            |
|---------------------------------------------|---------------|
| enumerate                                   | 同时拿索引和值       |
| zip                                         | 并行迭代多个可迭代     |
| heapq                                       | 堆/优先队列（小顶）    |
| bisect                                      | 有序list二分查找/插入 |
| functools.lru_cache                         | 记忆化搜索         |
| itertools.product/permutations/combinations | 笛卡尔积/排列/组合    |
| collections.Counter/defaultdict/deque       | 计数/默认字典/双端队列  |

十、考试技巧

- 先看数据规模选算法：n≤20可暴力，n≤1000可O(n²)，n≤10⁶需O(nlogn)。
- 边界先想清：n=0, n=1, 全相同, 全不同。
- 调试用小数据手算对比；大数据用断言定位。
- 数字字符串可能有前导零，需要数值比较时先转int。